Heterogeneous Graph Temporal Fusion Transformer for Time Series Forecasting in Multiphysics Systems

Anonymous Author(s)
Affiliation
Address
email

Abstract

Existing Transformer-based methods effectively capture multivariate time series dependencies, while pre-trained large models excel in generalizing across forecasting tasks, albeit typically in single-physics fields. Spatial-temporal approaches integrate graph structures to enhance forecasting, but often fall short in modeling the complexity of heterogeneous fields with diverse inter-variable interactions. These models also lack a comprehensive understanding of the underlying physical mechanisms in multiphysics fields, leading to dimensional inconsistencies. To address these challenges, we propose a pre-training and fine-tuning paradigm tailored for spatially and temporally structured physical environments. By tokenizing observation points and generating embeddings that capture temporal patterns and spatial correlations, our model, the heterogeneous graph temporal fusion transformer (HGTFT), effectively integrates multivariable dependencies and relational semantics. We also design a task pipeline with specialized loss functions and training strategies. Applied to temperature, flow, and energy-related field data in building environments, our method demonstrates strong zero-shot generalization and achieves substantial accuracy improvements through few-shot fine-tuning with domain-specific data.

1 Introduction

Time-series prediction in multiphysics systems is commonly handled by constructing separate models for each variable or group, using methods like Long Short-Term Memory (LSTM) [1], XGBoost [2], and ARIMA [3]. While effective in narrow domains, such models often lack generalization, relying heavily on project-specific data and ignoring broader contextual dependencies, which limits their adaptability across tasks. Large language models (LLMs), such as GPT-3 [4], demonstrate strong zero-shot and few-shot learning abilities in natural language processing. Some recent works explore their extension to time-series tasks [5, 6], yet they struggle to capture temporal patterns and domain-specific physical dynamics critical for accurate forecasting. Recent large time-series models (LTMs) [7, 8, 9, 10] exhibit promising generalization capabilities and perform well in multivariate forecasting across diverse domains. However, their focus on general temporal patterns limits the integration of static attributes, structural priors, and field-specific physical semantics, reducing their effectiveness in representing complex spatiotemporal interactions within physical fields.

To address the aforementioned limitations, we propose a novel time-series prediction framework built upon the heterogeneous graph temporal fusion transformer (HGTFT). Each token represents the state of a node at a specific time point, embedding both dynamic observations and static attributes. HGTFT jointly models temporal evolution and graph-structured spatial relationships, effectively capturing multi-scale dependencies within multiphysics systems. The framework adopts an encoder-decoder architecture. The encoder integrates temporal trajectories with heterogeneous spatial relations to

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

model system-wide physical dynamics. The decoder refines this representation to generate task-specific forecasts. This separation enables the encoder to learn generalizable physical patterns, while allowing the decoder to adapt flexibly to diverse downstream tasks. We design a multi-stage training pipeline that leverages the model architecture to capture both observed variability and underlying physical mechanisms, thereby enhancing adaptability across tasks. Our contributions are threefold: (1) We define the Heterogeneous Graph Forecasting in Multiphysics Systems problem, which applies to a broader range of real-world physical systems; (2) We propose the HGTFT framework and a multi-stage training pipeline designed to capture physical dynamics and improve generalization across diverse forecasting tasks; (3) Our method demonstrates strong performance on a self-constructed multiphysics building system dataset and other public benchmarks.

2 Related Work

Transformer for Time Series Forecasting. The Transformer model [11] has revolutionized time series forecasting by capturing long-range dependencies with attention mechanisms. Extensions like Informer [12] use probabilistic sparse attention for improved performance, while Frozen Pretrained Transformer [5] adapts pre-trained models from other domains, linking self-attention with principal component analysis. For multivariate forecasting, Crossformer [13] implements a two-stage attention mechanism to capture both temporal and cross-dimensional dependencies, and Temporal Fusion Transformer (TFT) [14] offers interpretable forecasting with mixed input handling. Recent approaches like TimeSiam [7] and Timer [8], leverage unlabeled data for representation learning. Unified training paradigms [9] enable single models to handle various forecasting tasks, while decoder-only models [10] enhance prediction efficiency. These efforts demonstrate the potential of pre-training to improve generalization and accuracy in time series forecasting.

Spatial-Temporal Forecasting. Graph Neural Networks (GNNs) have made strides in graph-based learning through structural and positional encoding. Approaches like LSPE [15] and NodeFormer [16] address scalability, while LETR [17] and Molecule Attention Transformer [18] apply Transformers to specialized tasks. For heterogeneous graphs, HDGT [19] and HAN [20] use hierarchical attention to capture diverse node and edge types. In spatiotemporal forecasting, Spacetimeformer [21] and heuristic graphs [22] model complex temporal-spatial sequences, while Graph Neural ODEs [23] incorporate differential equations for capturing dynamic temporal dependencies. Models like STSGCN [24] and STSGT [25] combine GCNs and Transformers to model synchronous spatial-temporal dependencies, applied to traffic and pandemic forecasting. Architectures such as HGT [26] and PromptST [27] leverage pretraining and adaptive tuning for heterogeneous, multi-attribute graph predictions. Similarly, UniST [28] employs prompt-based learning and extensive pre-training to enhance generalization in urban spatio-temporal prediction.

Large Language Models for Time Series. Large Language Models (LLMs) have been adapted for time series tasks, particularly in few-shot and zero-shot settings. TimeGPT-1 [29] reprograms LLMs for time series prediction by aligning embeddings with time-domain features, while Gruver [30] demonstrates zero-shot forecasting without fine-tuning. LLM4TS [31] and TIME-LLM [32] optimize LLMs for time series, improving adaptability to specialized datasets and temporal patterns. TimeCMA [33] introduces cross-modality alignment to enhance temporal understanding, and TimeChat [34] expands this to multimodal contexts, integrating temporal information for applications like video understanding. These studies highlight LLMs' potential as general-purpose forecasters, though challenges in temporal representation, data efficiency, and interpretability remain.

3 Problem Definition

General Spatial-Temporal Forecasting Problem. Spatial-temporal forecasting problems, such as those involving traffic networks, the COVID-19 pandemic, or power grids [35, 25, 36], can typically be formulated using a spatial network $G = (V, E, A)$, where V is the set of node vertices, E represents the set of edges, and A is the adjacency matrix describing relationships between nodes. The goal is often to predict future observations for a single node type with a single relationship type. Each node entity v_i in the graph is associated with a graph signal matrix $X(t)_G \in \mathbb{R}^{N \times F}$, where F is the number of features per node, and t denotes the time step. $X(t)_G$ captures the spatial network observations at time t, with each entry $X_{i,t}$ representing the feature vector of node v_i at time t. The task is to predict future spatial-temporal data by learning a mapping function $\mathcal{F}$ that maps historical

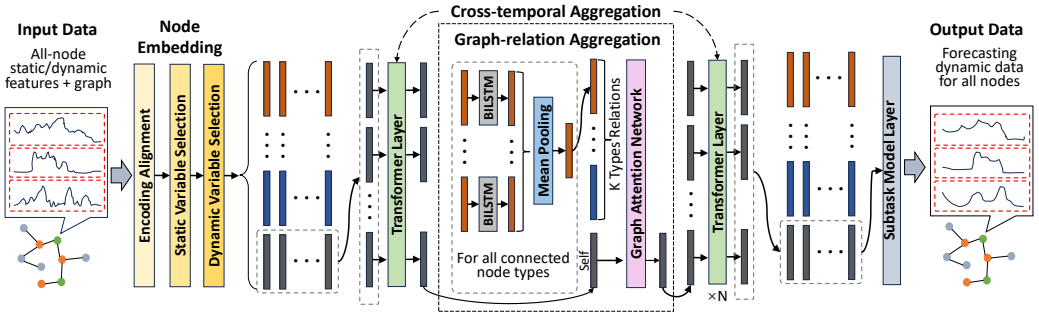

Figure 1: Architecture of the proposed HGTFT. The Fusion Layer converts heterogeneous data into unified-dimensional vectors, with colored bars indicating different object types. The Temporal Layer depicts processing for a single object, shared across all objects. The Graph Layer shows processing at one time point, replicated across all time steps.

series $\{X(t-T_{\text{past}}+1)_G, \ldots, X(t)_G\}$ to future observations $\{X(t+1)_G, \ldots, X(t+T_{\text{future}})_G\}$, where T_{past} is the length of historical data and T_{future} is the forecast horizon.

Extension to Heterogeneous Graph Forecasting in Multiphysics Systems. In contrast, our problem involves a more complex heterogeneous graph comprising multiple node types and relationships. Each node v_i is characterized by static attributes s_i and time-variant features, which are further grouped into: 1) variables known for both past and future x_i, 2) variables known only for the past z_i, and 3) the prediction variable y_i. The problem can be formulated as:

$$\hat{y}_{i,t+1:t+T_{\text{future}}} = \mathcal{F}(s_i, x_{i,t-T_{\text{past}}+1:t+T_{\text{future}}}, z_{i,t-T_{\text{past}}+1:t}, y_{i,t-T_{\text{past}}+1:t}, N(v_i)), \tag{1}$$

$$N(v_i) = \bigcup_{r_l \in R} N_l(v_i), \tag{2}$$

where $\hat{y}_{i,t+1:t+T_{\text{future}}}$ denotes the predicted target sequence for node v_i over the future horizon $[t+1, \ldots, t+T_{\text{future}}]$, and $\mathcal{F}$ is the learned forecasting function. $N(v_i)$ aggregates neighborhood information for node v_i across relation types r_l.

This extension is significant due to its ability to model complex multiphysics systems with diverse node types, features, and interrelationships, prevalent in real-world applications such as nuclear reactors, aerospace vehicles, biomedical devices, combined heat and power systems, and smart buildings. These systems necessitate advanced forecasting models capable of capturing intricate interdependencies and dynamic interactions across different physical domains. For illustrative examples and a discussion on the necessity of sophisticated modeling approaches, refer to Appendix A.

4 Model Architecture

The proposed HGTFT model, outlined in Figure 1, is designed for the previously defined problem by aggregating multi-dimensional data across static and dynamic node features within a heterogeneous graph structure. Features are aligned into unified embeddings per entity and time point, and these embeddings pass through neural layers that aggregate information across temporal and graph dimensions, resulting in fixed-dimension representations. The representations are then forwarded to task-specific model layer for dimension transformation tailored to each task.

Fusion Layer. Each node v_i is associated with static covariates s_i and time-varying features: future-known variables $x_{i,t}$, past-only variables $z_{i,t}$, and the target variable $y_{i,t}$. We first map all available inputs into a shared d-dimensional latent space and fuse them using a Variable Selection Network (VSN) inspired by TFT [14]. This produces a time-dependent node representation:

$$h_{i,t}^{\text{node}} = \text{VSN}\big(\text{Proj}(s_i),\ \text{Proj}(x_{i,t}),\ \text{Proj}(z_{i,t}),\ \text{Proj}(y_{i,t})\big), \tag{3}$$

where $\text{Proj}(\cdot)$ denotes a linear transformation that maps each input variable to a fixed-dimensional vector, ensuring compatibility with subsequent layers.

Temporal Layer. To capture temporal dependencies, we employ a unified Transformer-based architecture for all temporal processing layers. Specifically, we apply Transformer encoder layer to the historical node representations across time:

$$h_{i,t}^{\text{temp}} = \text{Transformer}\big(\{h_{i,t-T_{\text{past}}+1}^{\text{node}}, \ldots, h_{i,t}^{\text{node}}\}\big)_t, \tag{4}$$

where the output $h_{i,t}^{\text{temp}}$ denotes the temporally encoded representation of node v_i at time step t. To effectively capture temporal dependencies, Transformer-based temporal layers are positioned both between the fusion and graph layers, and following the graph layer. This design enables the model to capture temporal dependencies in the node representations before and after relational aggregation, facilitating deeper modeling of time-evolving dynamics across heterogeneous entities.

Graph Layer. We adopt the two-step relation-aware aggregation strategy from HetGNN [37], which separates message passing into *intra-relation* and *inter-relation* stages. Intra-relation aggregation independently encodes type-specific context by aggregating neighbor information within each relation type. The inter-relation step then fuses these intermediate features across relation types to form the final node representation. To enhance this process, we make two key modifications. First, we replace HetGNN's neural inter-relation fusion with a multi-head graph attention mechanism (GAT), enabling the model to assign adaptive importance to each relation type. Second, instead of grouping neighbors by node type, we explicitly distinguish edge relations and assign separate parameters per relation. This design captures finer-grained dependencies and mitigates semantic entanglement arising from heterogeneous physical or logical interactions. At each time step t, the system is modeled as a heterogeneous graph $\mathcal{G}_t = (V, E, R)$, where each node aggregates information from its multi-relational neighbors. For each relation type $r_\ell \in R$, we first compute:

$$h_{i,\ell}^{\text{agg}}(t) = \frac{1}{|N_\ell(v_i)|} \sum_{v_j \in N_\ell(v_i)} \text{BiLSTM}_\ell(h_{j,t}^{\text{temp}}), \tag{5}$$

$$\alpha_\ell^k = \text{softmax}\Big(\text{LeakyReLU}\left(a^{k\top}[W^k h_{i,t}^{\text{temp}} \| W^k h_{i,\ell}^{\text{agg}}(t)]\right)\Big), \tag{6}$$

$$h_{i,t}^{\text{graph}} = \frac{1}{K} \sum_{k=1}^{K} \sum_{\ell=1}^{L} \alpha_\ell^k W^k h_{i,\ell}^{\text{agg}}(t), \tag{7}$$

where K is the number of attention heads and $L = |R|$ is the number of relation types. $W^k \in \mathbb{R}^{d' \times d}$ is the learnable projection matrix for the k-th head, and $a^k \in \mathbb{R}^{2d'}$ is the shared attention vector for computing attention scores. This enables the model to selectively aggregate information from heterogeneous neighbor types in a multi-head attention manner.

Subtask Model Layer. To support diverse downstream forecasting objectives across heterogeneous entities, we adopt a modular subtask modeling framework. Each subtask shares a unified decoder architecture that transforms encoded representations into future predictions, as illustrated in Figure 9 in Appendix F.1.

The decoder leverages a masked multi-head attention (MHA) mechanism to align the encoded inputs $\{h_{i,t}\}_{t=t-T_{\text{past}}+1}^{t+T_{\text{future}}}$ with their respective future time steps. The output is then passed through a task-specific dense projection to generate the predicted dynamics $\{\hat{y}_{i,t'}\}_{t'=t+1}^{t+T_{\text{future}}}$:

$$\{\hat{y}_{i,t'}\}_{t'=t+1}^{t+T_{\text{future}}} = \text{Dense}(\text{MHA}(\{h_{i,t}\}_{t=t-T_{\text{past}}+1}^{t+T_{\text{future}}})). \tag{8}$$

To ensure stable information flow and consistent representation, the decoder incorporates Gated Residual Networks (GRNs), gating mechanisms, and Add & Norm layers.

The HGTFT framework has been validated for convergence on a simplified example in Appendix A.1.

5 Model Training Methodology

To forecast spatio-temporal dynamics in complex multiphysics systems, we design a targeted HGTFT training pipeline that sequentially combines self-supervised learning, multi-task training, and subtask fine-tuning to progressively improve modeling of heterogeneity and physical dependencies.

5.1 Multi-instance Normalization

Normalization is critical for improving model stability, convergence, and generalization. However, standard methods often fall short in our setting due to large intra-type variability (e.g., cooling loads significantly differ by room size) resulting in suboptimal gradient updates and biased loss weighting. To address this, we propose Multi-Instance Normalization. For each variable type j and instance i, we compute the time-series min and max values, then aggregate these across instances to derive the 10th percentile of minima and 90th percentile of maxima. These are used as normalization bounds:

$$\tilde{v}_{i,j}(t) = \frac{v_{i,j}(t) - \text{Percentile}_{10}(\{\min_t v_{i,j}(t)\}_i)}{\text{Percentile}_{90}(\{\max_t v_{i,j}(t)\}_i) - \text{Percentile}_{10}(\{\min_t v_{i,j}(t)\}_i)}. \tag{9}$$

This method ensures consistent scaling across instances of the same object type, improving learning dynamics and overall prediction accuracy. Further comparisons are provided in Appendix H.

5.2 Self-supervised Learning

Training the HGTFT model requires strategies that effectively encode temporal and relational dynamics. Self-supervised learning (SSL) offers a scalable approach by utilizing unlabeled spatio-temporal data via pretext tasks with pseudo-labels. Common tasks such as masked prediction and contrastive learning have demonstrated success in both graph and time-series domains [38, 39, 40]. We formulate SSL as the joint optimization of the foundation model f_θ and auxiliary heads p_ϕ on an unlabeled dataset D_1:

$$(\theta^*, \phi^*) = \arg\min_{\theta,\phi} L_{\text{ssl}}(f_\theta, p_\phi, D_1), \tag{10}$$

where L_{ssl} combines two tasks to capture both temporal dependencies and structural relationships.

Masked Time-Series Modeling. Following [41], portions of the input sequence are masked and reconstructed using Mean Squared Error (MSE) loss.

Masked Edge Modeling. A subset of graph edges are masked, and the model predicts them using Binary Cross-Entropy (BCE) loss, distinguishing between true and randomly sampled negative edges.

Both loss formulations are provided in Appendix E.1. To balance the tasks, we adopt an alternating training strategy: the two SSL tasks switch throughout training, beginning and ending with time-series modeling. This highlights sequence learning while integrating relational understanding.

5.3 Multi-task Supervised Learning

Building on the pre-trained foundation model f_{θ^*} from SSL, we introduce a multi-task supervised learning (MTSL) framework to further fine-tune parameters θ^{**} with task-specific heads q_ψ:

$$(\theta^{**}, \psi^*) = \arg\min_{\theta^*,\psi} L_{\text{MTSL}}(f_{\theta^*}, q_\psi, D_2, Y), \tag{11}$$

where D_2 is the labeled spatio-temporal dataset and Y denotes the task labels. Instead of simultaneous MTSL, which scales poorly with task count, we adopt a sequential training strategy that optimizes tasks one-by-one, reducing memory usage and promoting convergence in imbalanced multiphysics settings [42, 43].

Convergence Criterion. To determine when to stop sequential training, we monitor the relative loss change for each task i at iteration k:

$$\Delta L_{\text{task},i} = \frac{|L^{(k)}_{\text{task},i} - L^{(k-1)}_{\text{task},i}|}{L^{(k-1)}_{\text{task},i}}, \quad \Delta L_{\text{avg}} = \frac{1}{N}\sum_{i=1}^{N} \Delta L_{\text{task},i}, \tag{12}$$

where $L^{(k)}_{\text{task},i}$ is the loss of task i at iteration k, and N is the number of tasks. Training is considered converged when the average change ΔL_{avg} falls below a threshold (e.g., 2%).

Metrics. To align predictions with physical consistency, we augment the MSE loss with three terms: (1) Reasonableness Checks Score (RCS) discourages predictions that contradict domain-specific operational constraints or physical laws Appendix F.4; (2) Correlation-Based Score (CRS) promotes consistency with correlation patterns in real-world multivariate time-series data Appendix F.5; (3) Frequency Domain Similarity (FDS) aligns the spectral characteristics of predictions with those of the ground truth, using Fourier-based analysis Appendix F.6. The total loss for task i is:

$$L_{\text{task},i} = a_1 L_{\text{MSE}} + a_2 L_{\text{RCS}} + a_3 L_{\text{CRS}} + a_4 L_{\text{FDS}}, \tag{13}$$

where a_1, a_2, a_3, a_4 denote the respective weights of each loss component.

We adopt a hard parameter sharing scheme with a shared encoder and task-specific decoders, enabling both generalization and specialization. The complete MTSL training pipeline is organized into three stages, each with distinct settings and loss weights to support progressive specialization, as detailed in Appendix F.7.

5.4 Subtask Fine-tuning

The subtask fine-tuning process consists of two stages: task fine-tuning and project-specific fine-tuning, each targeting different adaptation needs.

Task fine-tuning adapts the pre-trained model to specific forecasting tasks. In this stage, the shared encoder layers are frozen, and only task-specific parameters are updated. This enhances task performance and serves as a pre-adaptation step for downstream applications.

Project-specific fine-tuning focuses on adapting the model to real-world scenarios with limited labeled data. Only lightweight components, such as the dense projection layer, are updated to align with new data distributions while retaining general representations learned during pretraining.

6 Experiments

We evaluate HGTFT on three dataset types: standard time series, graph-structured spatiotemporal data, and a multiphysics building system dataset. Baselines include traditional machine learning, deep learning models, and large model-based approaches. Evaluation metrics follow established practices for each dataset type. Ablation studies and further analyses are provided in later sections.

6.1 Datasets

Standard Time-Series Forecasting Datasets

We employ several widely-used time-series forecasting benchmarks, including ETT, Weather, Electricity, Traffic and Exchange, to assess general temporal prediction capabilities. These datasets are publicly available in [44] and have been extensively used in prior forecasting studies.

Graph-Structured Spatiotemporal Datasets

To evaluate the model's capacity to incorporate relational information, we conduct spatiotemporal forecasting on graph-structured datasets, including PeMSD4 and PeMSD8 traffic data from Caltrans PeMS [45], and COVID-19 case data from Johns Hopkins University [46] and the New York Times [47]. Each node is associated with a time series, and the graph structures encode spatial dependencies.

Multiphysics Building System Dataset

We evaluate our model on the Multiphysics Building System Dataset (MBS), which contains multivariate time-series data covering key variables across diverse objects. The dataset includes heterogeneous graphs that encode physical relationships among system elements, providing a complex and representative setting closely aligned with the challenges addressed in this work. The MBS dataset integrates data from 500 real-world buildings and 5,000 simulated buildings, the latter generated using EnergyPlus [48], a simulation tool developed by the U.S. Department of Energy. Real-world data undergo both algorithmic and expert-driven quality control, while simulated data are validated through scenario plausibility checks. All time series are sampled at 15-minute intervals. Further

Table 1: Performance on standard time-series forecasting and spatiotemporal datasets. ETT results are averaged over four subsets: ETTh1, ETTh2, ETTm1, and ETTm2. COVID-19 (JHU): daily infection counts from 83 Michigan counties; COVID-19 (NYT): daily death counts from 50 U.S. states. All models are trained or fine-tuned on 10% of each dataset. Best results are in **bold**, second best are underlined.

Dataset	Metric	LSTM	Autoformer	TFT	HTGNN	STD-MAE	TimesFM	MOIRAI	LLMTime	Time-LLM	HGTFT (Ours)
ETT	MSE	0.589	0.465	0.400	0.455	0.480	0.421	**0.391**	0.575	0.408	0.425
	MAE	0.597	0.459	0.412	0.484	0.534	0.437	**0.404**	0.577	0.428	0.441
Weather	MSE	0.332	0.338	0.292	0.335	0.393	0.299	0.259	0.345	**0.237**	0.299
	MAE	0.363	0.382	0.311	0.366	0.383	0.321	0.287	0.412	**0.264**	0.334
Electricity	MSE	0.268	0.227	0.239	0.263	0.257	0.245	0.192	0.276	**0.163**	0.219
	MAE	0.365	0.338	0.318	0.358	0.394	0.330	0.295	0.390	**0.264**	0.317
Traffic	MSE	0.804	0.628	0.646	0.552	0.596	0.521	0.620	0.813	**0.383**	0.481
	MAE	0.509	0.379	0.398	0.389	0.433	0.344	0.336	0.498	**0.264**	0.350
ILI	MSE	4.753	3.125	3.343	4.365	3.894	2.435	1.573	2.868	**1.437**	2.432
	MAE	1.580	1.168	1.281	1.550	1.489	1.021	0.935	1.047	**0.805**	1.077
PeMSD4	MAE	32.48	32.39	31.32	21.01	**17.85**	32.57	33.31	33.69	32.23	19.94
	RMSE	55.51	53.19	48.37	36.44	**29.72**	55.94	55.51	52.49	52.18	32.16
PeMSD8	MAE	24.98	25.56	24.63	18.22	**13.67**	23.93	24.03	26.68	27.74	16.43
	RMSE	41.71	41.65	39.74	27.04	**22.62**	42.41	42.49	43.94	40.01	25.08
COVID-19 (JHU)	MAE	122.42	115.77	121.81	46.24	47.75	99.75	105.74	115.37	95.01	**41.54**
	RMSE	232.11	198.67	261.77	102.73	**92.62**	216.63	234.24	216.74	201.14	94.38
COVID-19 (NYT)	MAE	70.59	62.37	71.36	31.16	26.69	57.07	81.05	72.24	83.17	**25.69**
	RMSE	139.18	133.35	158.93	75.98	72.98	113.45	134.57	157.31	146.45	**65.64**

Table 2: Average forecasting performance over 10 runs on 50 randomly selected building cases from the multiphysics system dataset. Best results in **bold**, second best underlined.

Metric	LSTM	Autoformer	TFT	HTGNN	STD-MAE	TimesFM	MOIRAI	LLMTime	Time-LLM	HGTFT zero-shot	HGTFT few-shot
MSE	0.0049	0.0053	0.0044	0.0048	0.0051	0.0064	0.0072	0.0093	0.0085	0.0027	**0.0023**
RCS	0.0376	0.0298	0.0152	0.0133	0.0206	0.0287	0.0243	0.0320	0.0307	**0.0012**	0.0029
CRS	0.476	0.521	0.503	0.386	0.280	0.531	0.526	0.553	0.569	0.312	**0.258**
FDS	0.438	0.455	0.429	0.476	0.493	0.368	0.380	0.578	0.514	0.405	**0.292**

details of the MBS dataset can be found in Appendix B, and a subset of the dataset is available in (https://drive.google.com/drive/folders/1fOG6SdFXXdJ0LtaELQA6o7obRxgTBfpg?usp=sharing).

6.2 Baselines

We compare our approach against a diverse set of baselines, encompassing traditional machine learning models, graph-based methods, and recent advancements in large pre-trained time-series models. Traditional models such as LSTM [1], as well as more recent architectures like Autoformer [49], forecast each variable independently without incorporating structural information. TFT [14] integrates static covariates with dynamic time-series inputs for multivariate forecasting. HTGNN [50] and STD-MAE [51] operate on graph-structured time-series data, with heterogeneous and homogeneous structures, respectively. Recent developments in large pre-trained models have shown significant promise. TimesFM [10] and MOIRAI [9] represent general pre-trained time-series models. LLM-based approaches, including Time-LLM [32] and LLMTimed [30], leverage large language models for time-series prediction. Further details on baseline selection are provided in Appendix G.

6.3 Main Results

We evaluate our method and selected baselines on standard time-series and spatiotemporal forecasting datasets, using MSE and MAE for the former, and MAE and RMSE for the latter, with results presented in Table 1. Our method achieves performance comparable to state-of-the-art models on standard benchmarks, particularly for spatiotemporal forecasting tasks. Our method addresses time-series forecasting in multiphysics systems with heterogeneous entities and interacting physical

Table 3: Results of ablation studies on model architecture modifications and simplifications.

Metric	HGTFT	Fusion: dense	Graph: removed	Graph: HTGNN	Temporal: removed	Temporal: Pre-G	Temporal: Post-G	Subtask: w/o GRU	Subtask: dense
MSE	**0.0027**	0.0053	0.0065	0.0032	0.0072	0.0063	0.0064	0.0048	0.0067
RCS	**0.0012**	0.0247	0.0343	0.0037	0.0157	0.0112	0.0103	0.0136	0.0297
CRS	**0.312**	0.523	0.355	0.338	0.632	0.536	0.529	0.462	0.544
FDS	**0.405**	0.417	0.414	0.416	0.589	0.512	0.515	0.496	0.580

processes. We evaluate HGTFT on the MBS dataset, which contains time-series data from diverse building operation scenarios. Each forecasting task uses the previous 7 days (672 time steps) to predict the next day (96 time steps), at 15 minute intervals. Due to large inter-variable scale differences, all metrics (MSE, RCS, CRS, and FDS) are computed on normalized values. For each of 50 randomly selected building cases, six months of data are randomly chosen for training or fine-tuning (for baselines and HGTFT few-shot), and the remaining six months for testing. HGTFT is evaluated under two settings: (i) **zero-shot**, where the pre-trained model predicts future values for unseen cases without access to their data, and (ii) **few-shot**, where limited case-specific data is used for adaptation. Each experiment is repeated 10 times with different seeds, and results are averaged. As shown in Table 2, HGTFT achieves strong zero-shot performance and further improves with few-shot fine-tuning, highlighting its generalization to complex physical environments.

6.4 Ablation Study

We perform ablation studies to evaluate the contribution of key HGTFT components. For static-dynamic fusion, VSNs are replaced with dense layers. For structural modeling, we either remove the graph encoder or replace it with HTGNN-style aggregation. For temporal modeling, we reduce Transformer depth, test a single layer placed before or after the graph layer (denoted as Pre-G and Post-G), or remove it entirely. The subtask layer is simplified by removing GRU and Add & Norm units or keeping only a dense projection. Results on MSE, RCS, CRS, and FDS are reported in Table 3. Model scaling results (see Appendix D) indicate that performance improves with size up to 310M parameters, beyond which marginal gains diminish, suggesting 310M as a favorable balance between model capacity and efficiency.

6.5 Further Analysis

To evaluate the model's capacity to capture multiphysics interactions, Figure 2 visualizes predicted temperature fields on a sample floor at a selected time slice. The HGTFT model using full inputs (dynamic, static, and graph data) accurately reconstructs spatial temperature patterns. In contrast, the variant excluding static features (e.g., zone type, orientation) and spatial adjacency yields less coherent results, underscoring the importance of incorporating static and graph information.

To evaluate generalization under distributional shifts, we test rare control actions on the selected day—such as activating 2 or 4 chillers—that deviate from typical operating conditions. As shown in Figure 3, the model trained with multi-metric supervision yields consistent and physically plausible

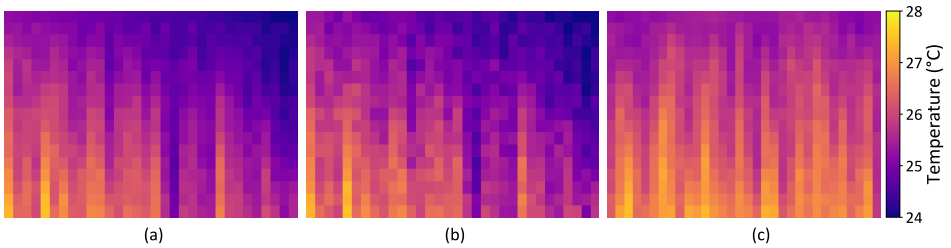

Figure 2: Predicted temperature fields for a sample floor at a selected time slice. (a) ground truth, (b) prediction by HGTFT using full input, and (c) prediction without static or graph information.

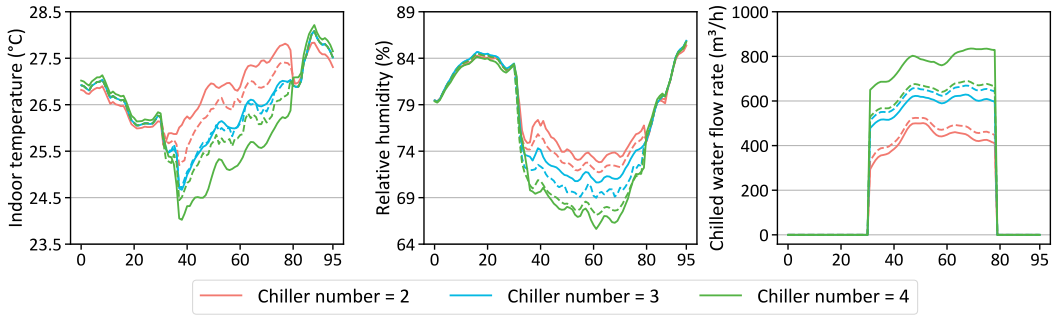

Figure 3: Predicted system responses to changes in control actions (solid: multi-metric training model; dashed: MSE-only training model).

responses across chilled water flow, indoor temperature, and humidity. Compared to the MSE-only model, it exhibits greater sensitivity to large-scale control changes. For instance, when increasing the number of chillers from 3 to 4, the multi-metric model captures the expected rise in chilled water flow and drop in indoor temperature, whereas the MSE-only model produces overly smooth or saturated outputs, with predictions under 4-chiller operation remaining too close to those under 3-chiller conditions—indicating weaker generalization to rare or unseen control patterns. Metric ablation further shows that removing RCS, CRS, or FDS increases their respective evaluation scores from 0.0012 to 0.0134 (RCS), 0.312 to 0.384 (CRS), and 0.405 to 0.423 (FDS), while other metrics remain nearly unchanged. This highlights RCS as the most impactful among the three.

To further validate our methodology, we conduct extensive analyses. First, we compare normalization strategies using CV-RMSE across variable types and observe that the proposed Multi-Instance Normalization consistently improves optimization stability and generalization over Min-Max and Z-Score methods (Appendix H). For self-supervised learning, we evaluate three training strategies and find that while simultaneous task training impairs forecasting quality, our alternating task training method effectively balances time-series and relational representations with lower losses (Appendix J.1). In supervised fine-tuning, sequential multi-task training yields stable convergence, and reducing either task types or data coverage leads to moderate performance drops, highlighting the importance of task and case diversity (Appendix J.2). Input/output horizon analysis reveals that longer input patches enhance short- to mid-term forecasting accuracy, demonstrating the value of extended temporal context (Appendix J.3).

7 Conclusion

This paper proposes a unified framework for time series forecasting in multiphysics systems, grounded in a clear problem formulation. Our key contributions include a novel tokenization strategy for heterogeneous entities, embedding and aggregation algorithms (HGTFT) that jointly capture temporal and relational dependencies, as well as specialized training techniques, evaluation metrics, and task design strategies. While our method achieves performance comparable to state-of-the-art models on widely used benchmarks, it shows clear advantages in complex, realistic scenarios involving heterogeneous multiphysics systems. We validate this on a new dataset designed to reflect diverse object types, physical interactions, and control scenarios in building systems, where our model demonstrates strong zero-shot performance and further gains through few-shot finetuning, confirming its suitability for real-world applications.

Despite promising results, several challenges remain. The dataset requires broader coverage of scenarios and components to support generalization across diverse multiphysics environments. While few-shot finetuning improves performance under short-term or similar conditions, it may degrade over longer horizons or under significant distributional shifts. Future work will explore selective adaptation strategies that automatically identify relevant finetuning targets. The current need for retraining across different temporal granularities and spans also highlights the importance of improving model flexibility. Further refinement of normalization strategies is needed to ensure consistency across heterogeneous variables. Addressing these issues will enhance the model's robustness and scalability in real-world physical systems.

References

[1] S Hochreiter. Long short-term memory. *Neural Computation MIT-Press*, 1997.

[2] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

[3] Brian K Nelson. Time series analysis using autoregressive integrated moving average (arima) models. *Academic emergency medicine*, 5(7):739–744, 1998.

[4] Tom B Brown. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

[5] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems*, 36:43322–43355, 2023.

[6] Chenxi Liu, Sun Yang, Qianxiong Xu, Zhishuai Li, Cheng Long, Ziyue Li, and Rui Zhao. Spatial-temporal large language model for traffic prediction. *arXiv preprint arXiv:2401.10134*, 2024.

[7] Jiaxiang Dong, Haixu Wu, Yuxuan Wang, Yunzhong Qiu, Li Zhang, Jianmin Wang, and Mingsheng Long. Timesiam: A pre-training framework for siamese time-series modeling. *arXiv preprint arXiv:2402.02475*, 2024.

[8] Yong Liu, Haoran Zhang, Chenyu Li, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. Timer: Generative pre-trained transformers are large time series models. In *Forty-first International Conference on Machine Learning*, 2024.

[9] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. Unified training of universal time series forecasting transformers. *arXiv preprint arXiv:2402.02592*, 2024.

[10] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. *arXiv preprint arXiv:2310.10688*, 2023.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need.(nips), 2017. *arXiv preprint arXiv:1706.03762*, 10:S0140525X16001837, 2017.

[12] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 11106–11115, 2021.

[13] Yunhao Zhang and Junchi Yan. Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *The eleventh international conference on learning representations*, 2023.

[14] Bryan Lim, Sercan Ö Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

[15] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform badly for graph representation? *Advances in neural information processing systems*, 34:28877–28888, 2021.

[16] Qitian Wu, Wentao Zhao, Zenan Li, David P Wipf, and Junchi Yan. Nodeformer: A scalable graph structure learning transformer for node classification. *Advances in Neural Information Processing Systems*, 35:27387–27401, 2022.

[17] Yifan Xu, Weijian Xu, David Cheung, and Zhuowen Tu. Line segment detection using transformers without edges. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4257–4266, 2021.

[18] Łukasz Maziarka, Tomasz Danel, Sławomir Mucha, Krzysztof Rataj, Jacek Tabor, and Stanisław Jastrzębski. Molecule attention transformer. *arXiv preprint arXiv:2002.08264*, 2020.

[19] Xiaosong Jia, Penghao Wu, Li Chen, Yu Liu, Hongyang Li, and Junchi Yan. Hdgt: Heterogeneous driving graph transformer for multi-agent trajectory prediction via scene encoding. *IEEE transactions on pattern analysis and machine intelligence*, 2023.

[20] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. Heterogeneous graph attention network. In *The world wide web conference*, pages 2022–2032, 2019.

[21] Jake Grigsby, Zhe Wang, Nam Nguyen, and Yanjun Qi. Long-range transformers for dynamic spatiotemporal forecasting. *arXiv preprint arXiv:2109.12218*, 2021.

[22] Wei Shao, Zhiling Jin, Shuo Wang, Yufan Kang, Xiao Xiao, Hamid Menouar, Zhaofeng Zhang, Junshan Zhang, and Flora Salim. Long-term spatio-temporal forecasting via dynamic multiple-graph attention. *arXiv preprint arXiv:2204.11008*, 2022.

[23] Michael Poli, Stefano Massaroli, Junyoung Park, Atsushi Yamashita, Hajime Asama, and Jinkyoo Park. Graph neural ordinary differential equations. *arXiv preprint arXiv:1911.07532*, 2019.

[24] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. Spatial-temporal synchronous graph convolutional networks: A new framework for spatial-temporal network data forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 914–921, 2020.

[25] Soumyanil Banerjee, Ming Dong, and Weisong Shi. Spatial–temporal synchronous graph transformer network (stsgt) for covid-19 forecasting. *Smart Health*, 26:100348, 2022.

[26] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. Heterogeneous graph transformer. In *Proceedings of the web conference 2020*, pages 2704–2710, 2020.

[27] Zijian Zhang, Xiangyu Zhao, Qidong Liu, Chunxu Zhang, Qian Ma, Wanyu Wang, Hongwei Zhao, Yiqi Wang, and Zitao Liu. Promptst: Prompt-enhanced spatio-temporal multi-attribute prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, pages 3195–3205, 2023.

[28] Yuan Yuan, Jingtao Ding, Jie Feng, Depeng Jin, and Yong Li. Unist: A prompt-empowered universal model for urban spatio-temporal prediction. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 4095–4106, 2024.

[29] Azul Garza, Cristian Challu, and Max Mergenthaler-Canseco. Timegpt-1. *arXiv preprint arXiv:2310.03589*, 2023.

[30] Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew G Wilson. Large language models are zero-shot time series forecasters. *Advances in Neural Information Processing Systems*, 36, 2024.

[31] Ching Chang, Wei-Yao Wang, Wen-Chih Peng, and Tien-Fu Chen. Llm4ts: Aligning pre-trained llms as data-efficient time-series forecasters. *arXiv preprint arXiv:2308.08469*, 2024.

[32] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728*, 2023.

[33] Chenxi Liu, Qianxiong Xu, Hao Miao, Sun Yang, Lingzheng Zhang, Cheng Long, Ziyue Li, and Rui Zhao. Timecma: Towards llm-empowered time series forecasting via cross-modality alignment. *arXiv preprint arXiv:2406.01638*, 2024.

[34] Shuhuai Ren, Linli Yao, Shicheng Li, Xu Sun, and Lu Hou. Timechat: A time-sensitive multimodal large language model for long video understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14313–14323, 2024.

[35] Shengnan Guo, Youfang Lin, Ning Feng, Chao Song, and Huaiyu Wan. Attention based spatial-temporal graph convolutional networks for traffic flow forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 922–929, 2019.

[36] Jiansong Liu, Yan Kang, Hao Li, Haining Wang, and Xuekun Yang. Stghtn: Spatial-temporal gated hybrid transformer network for traffic flow forecasting. *Applied Intelligence*, 53(10):12472–12488, 2023.

[37] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. Heterogeneous graph neural network. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 793–803, 2019.

[38] Veenu Rani, Syed Tufael Nabi, Munish Kumar, Ajay Mittal, and Krishan Kumar. Self-supervised learning: A succinct review. *Archives of Computational Methods in Engineering*, 30(4):2761–2775, 2023.

[39] Yaochen Xie, Zhao Xu, Jingtun Zhang, Zhengyang Wang, and Shuiwang Ji. Self-supervised learning of graph neural networks: A unified review. *IEEE transactions on pattern analysis and machine intelligence*, 45(2):2412–2429, 2022.

[40] Kexin Zhang, Qingsong Wen, Chaoli Zhang, Rongyao Cai, Ming Jin, Yong Liu, James Y Zhang, Yuxuan Liang, Guansong Pang, Dongjin Song, et al. Self-supervised learning for time series analysis: Taxonomy, progress, and prospects. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.

[41] George Zerveas, Srideepika Jayaraman, Dhaval Patel, Anuradha Bhamidipaty, and Carsten Eickhoff. A transformer-based framework for multivariate time series representation learning. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 2114–2124, 2021.

[42] Simon Vandenhende, Stamatios Georgoulis, Wouter Van Gansbeke, Marc Proesmans, Dengxin Dai, and Luc Van Gool. Multi-task learning for dense prediction tasks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3614–3633, 2021.

[43] Jun Yu, Yutong Dai, Xiaokang Liu, Jin Huang, Yishan Shen, Ke Zhang, Rong Zhou, Eashan Adhikarla, Wenxuan Ye, Yixin Liu, et al. Unleashing the power of multi-task learning: A comprehensive survey spanning traditional, deep, and pretrained foundation model eras. *arXiv preprint arXiv:2404.18961*, 2024.

[44] W Haixu, X Jiehui, J Wang, and L Mingsheng. Decomposition transformers with auto-correlation for long-term series forecasting, 2022.

[45] Chao Chen, Karl F Petty, Alexander Skabardonis, Pravin P Varaiya, and Zhigang Jia. Freeway performance measurement system: mining loop detector data. In *Proceedings of the 80th Annual Meeting of the Transportation Research Board*. Citeseer, 2001.

[46] Ensheng Dong, Hongru Du, and Lauren Gardner. An interactive web-based dashboard to track covid-19 in real time. *The Lancet infectious diseases*, 20(5):533–534, 2020.

[47] Covid-19 data repository by the new york times. `https://github.com/nytimes/covid-19-data`. Accessed: 2025-04-27.

[48] US DOE. Application guide for ems energy management system–user guide, 2015.

[49] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems*, 34:22419–22430, 2021.

[50] Yujie Fan, Mingxuan Ju, Chuxu Zhang, and Yanfang Ye. Heterogeneous temporal graph neural network. In *Proceedings of the 2022 SIAM International Conference on Data Mining (SDM)*, pages 657–665. SIAM, 2022.

[51] Haotian Gao, Renhe Jiang, Zheng Dong, Jinliang Deng, Yuxin Ma, and Xuan Song. Spatial-temporal-decoupled masked pre-training for spatiotemporal forecasting. *arXiv preprint arXiv:2312.00516*, 2023.

[52] Haoyu Han, Mengdi Zhang, Min Hou, Fuzheng Zhang, Zhongyuan Wang, Enhong Chen, Hongwei Wang, Jianhui Ma, and Qi Liu. Stgcn: a spatial-temporal aware graph learning method for poi recommendation. In *2020 IEEE International Conference on Data Mining (ICDM)*, pages 1052–1057. IEEE, 2020.

[53] Mengzhang Li and Zhanxing Zhu. Spatial-temporal fusion graph neural networks for traffic flow forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 4189–4196, 2021.

[54] Zheng Fang, Qingqing Long, Guojie Song, and Kunqing Xie. Spatial-temporal graph ode networks for traffic flow forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 364–373, 2021.

[55] Jinliang Deng, Xiusi Chen, Renhe Jiang, Xuan Song, and Ivor W Tsang. St-norm: Spatial and temporal normalization for multi-variate time series forecasting. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*, pages 269–278, 2021.

[56] Shiyong Lan, Yitong Ma, Weikang Huang, Wenwu Wang, Hongyu Yang, and Pyang Li. Dstagnn: Dynamic spatial-temporal aware graph neural network for traffic flow forecasting. In *International conference on machine learning*, pages 11906–11917. PMLR, 2022.

[57] Jiawei Jiang, Chengkai Han, Wayne Xin Zhao, and Jingyuan Wang. Pdformer: Propagation delay-aware dynamic long-range transformer for traffic flow prediction. In *Proceedings of the AAAI conference on artificial intelligence*, volume 37, pages 4365–4373, 2023.

[58] Hangchen Liu, Zheng Dong, Renhe Jiang, Jiewen Deng, Jinliang Deng, Quanjun Chen, and Xuan Song. Staeformer: spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting. *arXiv preprint arXiv:2308.10425*, 2023.

A Description of Typical Problems

We present two examples to highlight the significance of extending the problem within the context of multiphysics systems (e.g., building systems). The first example involves a relatively simple dynamic system model, which begins with the fan coil unit (FCU) in relation to the space and cooling source. We then extend to a more complex system, which includes multiple types of objects and relationships, with each type of object potentially having a large number of instances.

A.1 Example 1: Heat Exchange in Fan Coil Unit (FCU)

In the FCU, heat exchange occurs between air and water, and this process can be modeled using differential equations. Let's define the problem and derive the equations step by step.

Problem Definition

The heat exchange process involves the flow of air and water through the FCU, where air absorbs heat from the water and vice versa. The temperature dynamics for air and water are described as follows:

We first describe the air temperature dynamics. The rate of change of air temperature is governed by the following equation:

$$\dot{m}_{\text{air}} c_{\text{air}} \frac{dT_{\text{air}}(t)}{dt} = \dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t)) - Q_{\text{heat,air}}(t) \tag{14}$$

where: - $\dot{m}_{\text{air}}$ is the mass flow rate of air (kg/s), - c_{air} is the specific heat capacity of air (J/kg·K), - $T_{\text{in,air}}(t)$ and $T_{\text{out,air}}(t)$ are the inlet and outlet air temperatures at time t (°C or K), - $Q_{\text{heat,air}}(t)$ is the heat exchanged between air and water at time t (W).

Next, we consider the water temperature dynamics. The change in water temperature over time can be described as:

$$\dot{m}_{\text{water}} c_{\text{water}} \frac{dT_{\text{water}}(t)}{dt} = Q_{\text{heat,water}}(t) - Q_{\text{water,out}}(t) \tag{15}$$

where: - $\dot{m}_{\text{water}}$ is the mass flow rate of water (kg/s), - c_{water} is the specific heat capacity of water (J/kg·K), - $T_{\text{water}}(t)$ is the water temperature at time t (°C or K), - $Q_{\text{heat,water}}(t)$ is the heat exchanged between air and water at time t (W), - $Q_{\text{water,out}}(t)$ is the heat lost by water to external factors at time t (W).

The heat exchange between air and water is modeled by the following equation:

$$Q_{\text{heat,air}}(t) = Q_{\text{heat,water}}(t) = h_{\text{air-water}} A_{\text{heat}} (T_{\text{air}}(t) - T_{\text{water}}(t)) \tag{16}$$

where: - $h_{\text{air-water}}$ is the heat transfer coefficient between air and water (W/m²·K), - A_{heat} is the heat exchange area (m²), - $T_{\text{air}}(t)$ and $T_{\text{water}}(t)$ are the air and water temperatures at time t (K).

Derivation of Differential Equations

Combining the heat exchange formulas with the temperature dynamics, we get a system of differential equations:

$$\dot{m}_{\text{air}} c_{\text{air}} \frac{dT_{\text{air}}(t)}{dt} = \dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t)) - h_{\text{air-water}} A_{\text{heat}} (T_{\text{air}}(t) - T_{\text{water}}(t)) \tag{17}$$

$$\dot{m}_{\text{water}} c_{\text{water}} \frac{dT_{\text{water}}(t)}{dt} = h_{\text{air-water}} A_{\text{heat}} (T_{\text{air}}(t) - T_{\text{water}}(t)) - Q_{\text{water,out}}(t) \tag{18}$$

Introducing Temperature Difference

To simplify the equations, introduce the temperature difference:

$$\Delta T(t) = T_{\text{air}}(t) - T_{\text{water}}(t) \tag{19}$$

Thus, the air temperature and water temperature can be expressed as:

$$T_{\text{air}}(t) = T_{\text{water}}(t) + \Delta T(t) \tag{20}$$

Substituting this into the differential equations, we get:

For the air temperature equation:

$$\dot{m}_{\text{air}} c_{\text{air}} \frac{d\Delta T(t)}{dt} = \dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t)) - h_{\text{air-water}} A_{\text{heat}} \Delta T(t) \tag{21}$$

For the water temperature equation:

$$\dot{m}_{\text{water}} c_{\text{water}} \frac{dT_{\text{water}}(t)}{dt} = h_{\text{air-water}} A_{\text{heat}} \Delta T(t) - Q_{\text{water,out}}(t) \tag{22}$$

Analytical Solution

For the temperature difference equation $\Delta T(t)$, we obtain:

$$\dot{m}_{\text{air}} c_{\text{air}} \frac{d\Delta T(t)}{dt} = \dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t)) - h_{\text{air-water}} A_{\text{heat}} \Delta T(t) \tag{23}$$

This is a first-order linear differential equation, which can be solved as:

$$\Delta T(t) = \frac{\dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t))}{h_{\text{air-water}} A_{\text{heat}}} \left(1 - e^{-\frac{h_{\text{air-water}} A_{\text{heat}}}{\dot{m}_{\text{air}} c_{\text{air}}} t}\right) \tag{24}$$

Using the initial condition $\Delta T_0 = T_{\text{air},0} - T_{\text{water},0}$, we obtain:

$$T_{\text{air}}(t) = T_{\text{water}}(t) + \frac{\dot{m}_{\text{air}} c_{\text{air}} (T_{\text{in,air}}(t) - T_{\text{out,air}}(t))}{h_{\text{air-water}} A_{\text{heat}}} \left(1 - e^{-\frac{h_{\text{air-water}} A_{\text{heat}}}{\dot{m}_{\text{air}} c_{\text{air}}} t}\right) \tag{25}$$

The analytical solution for the water temperature is:

$$T_{\text{water}}(t) = T_{\text{water},0} + \frac{h_{\text{air-water}} A_{\text{heat}} \Delta T_0}{\dot{m}_{\text{water}} c_{\text{water}}} \left(1 - e^{-\frac{h_{\text{air-water}} A_{\text{heat}}}{\dot{m}_{\text{air}} c_{\text{air}}} t}\right) - \frac{Q_{\text{water,out}}(t)}{\dot{m}_{\text{water}} c_{\text{water}}} \tag{26}$$

Challenges and Complexities

The heat lost by water, $Q_{\text{water,out}}$, is influenced by heat/cooling source objects, while $T_{\text{in,air}}$ and $T_{\text{out,air}}$ are connected to spaces. Both heat/cooling source objects and spaces have their own distinct features and dynamics. The primary challenge in modeling such a system lies in the complex coupling of air and water dynamics, as well as the interactions between multiple spaces and Fan Coil Units (FCUs). As the number of spaces and FCUs increases, the complexity of the system grows exponentially, making it increasingly difficult to derive a closed-form solution. Therefore, the ability to integrate multiple object types and relationships through neural network algorithms is a critical requirement for addressing such problems.

Numerical Simulation and Prediction Using HGTFT

In this study, we assign different values to the static parameters in the previously defined mathematical model and apply time-varying functions to the external variables, $Q_{\text{water,out}}(t)$, $T_{\text{in,air}}(t)$, and $T_{\text{out,air}}(t)$. Through numerical simulations, a dataset is generated, which is then used to train the HGTFT-based model. The objective of this training is to predict the temperature profiles $T_{\text{water}}(t)$ and $T_{\text{air}}(t)$ based on the temporal variations of the external variables and the given static parameters.

Figure 4 illustrates a numerical simulation where time-series values are generated under the assumption of sinusoidal variations for $Q_{\text{water,out}}(t)$, $T_{\text{in,air}}(t)$, and $T_{\text{out,air}}(t)$. The figure also shows the corresponding predictions of $T_{\text{water}}(t)$ and $T_{\text{air}}(t)$ obtained using the HGTFT model.

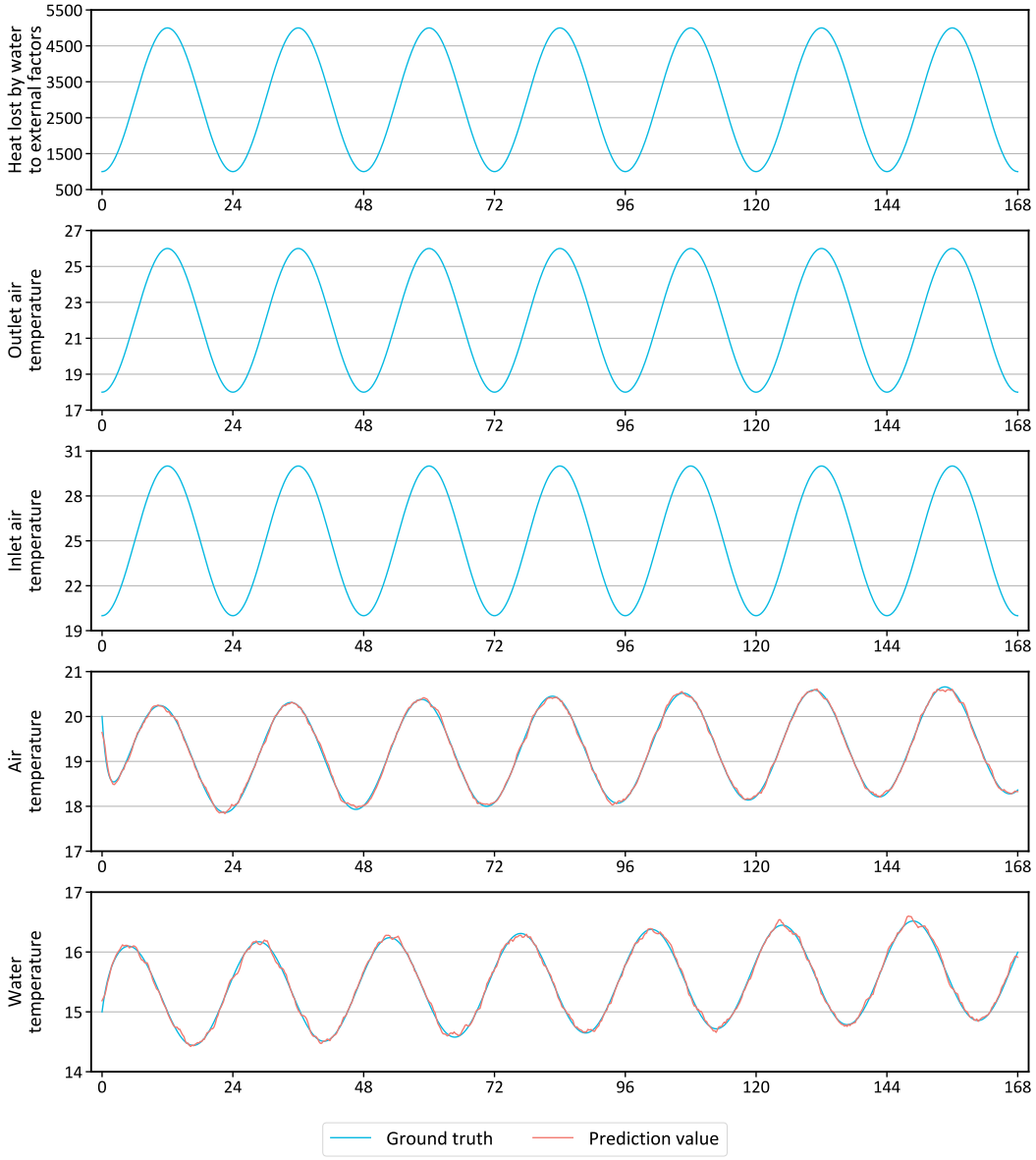

Figure 4: Numerical simulation for Example 1 and predictions using the HGTFT model.

A.2 Example 2: Forecasting in a Heterogeneous Multiphysics HVAC Network

This example highlights a forecasting task in a complex HVAC operation scenario involving heterogeneous entities with interconnected physical relationships. As shown in Table 4, the task spans multiple object types—such as environmental sensors, thermal zones, chillers, pumps, and air-side equipment—each with distinct static attributes and time-dependent dynamics. The temporal forecasting goal varies across objects, with certain variables provided as future inputs (e.g., control signals or setpoints), while others are to be predicted. This setup reflects real-world complexity where forecasting depends on both physical coupling (e.g., energy and fluid flow) and system control behavior.

Table 4: An example of an HVAC operation task, covering multiple object types—such as environment, general zone, chiller, chilled water pump (ACCCCP), cooling water pump (ACCCOP), cooling tower (ACCCOT), fan coil unit (ACATFC), and supply air fan (ACATFU)—along with their associated input and output variable types.

Object type	Input			Output
	Static attribute	Dynamic variable for the past	Dynamic attribute for the future	Dynamic attribute for the future
Environment		Outdoor temperature	Outdoor temperature	
GeneralZone	Area, volume, orientation	Indoor temperature, relative humidity		Indoor temperature, relative humidity
Chiller	Rated cooling capacity, rated power	Chilled water supply temperature, chilled water return temperature, chilled water flow rate		Chilled water supply temperature, chilled water return temperature, chilled water flow rate
ACCCCP	Rated power, rated flow rate, rated head	Operating status, operating power, flow rate	Operating status	Operating power, flow rate
ACCCOP	Rated power, rated flow rate, rated head	Operating status, operating power, flow rate	Operating status	Operating power, flow rate
ACCCOT	Rated power, rated air flow, number of fans, design outdoor wet-bulb temperature	Number of operating fans, air flow rate, leaving water temperature, water flow rate, leaving water temperature setpoint	Leaving water temperature Setpoint	Number of Operating fans, air flow rate, leaving water temperature, water flow rate
ACATFC	Rated power, rated air flow, rated chilled water flow rate	Supply air temperature, return air temperature, supply air temperature setpoint	Supply air temperature setpoint	Supply air temperature, return air temperature
ACATFU	Rated power, rated air flow	Fresh air flow rate, fan speed	Fan speed	Fresh air flow rate

B MBS Dataset Details

The definitions of objects and relationships in the building operation systems are derived from a publicly available, standardized data dictionary commonly used in the building automation domain. The training dataset primarily consists of HVAC-related data, incorporating both empirical data aggregated from diverse real-world deployments and synthetic data produced through a high-fidelity simulation environment. Figure 5 presents an example of a partial 3D visualization from the simulation setup, illustrating mappings between equipment and spatial zones, as well as detailed interconnections such as piping and ductwork.

B.1 Real Project Data

We have accumulated a substantial dataset from a multitude of real-world projects, encompassing various building subsystems such as energy management systems, security surveillance systems, equipment and facility management systems, and building automation systems. The dataset comprises a total of 1045 projects, with 508 projects containing relatively comprehensive information. The dataset contains about 5B tokens and 16B time points data.

B.2 Simulation Data

Compared to real-world project data, simulations can involve a much larger number of variables, including many that are difficult or even impossible to measure in the real world project but can be calculated in a simulation environment. Additionally, simulations allow for the alteration of many operating conditions, covering a much broader range of scenarios than real projects can achieve. Given the astronomical number of possible parameter combinations, it is necessary to reduce the number of generated simulation cases. This can be achieved by carefully selecting variable parameters and applying orthogonal testing to optimize the case generation process. We constructed a massive dataset of building energy simulations using EnergyPlus [48]. By systematically varying key building parameters across 12 diverse base building models, we generated approximately 5,000 simulation scenario cases. Each case provides high-resolution 15-minute data for a year, resulting in a dataset of over 80B tokens and 600B time points data.

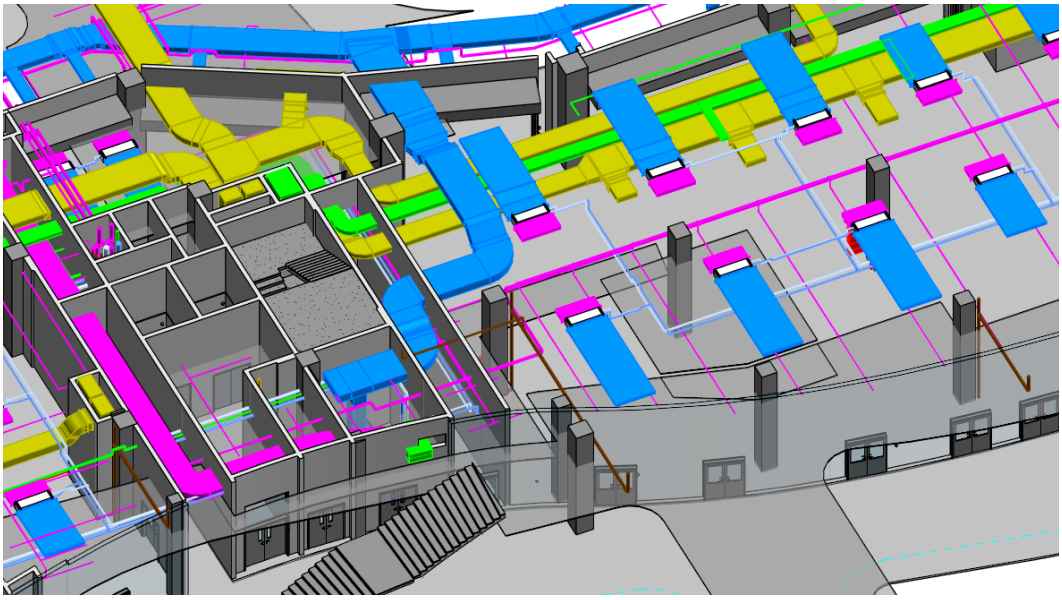

Figure 5: 3D illustration of a simulated building environment, showing spatial layout, service relationships between equipment and zones, and extensive duct and piping connections representing air and water flows in HVAC systems.

B.3 Comparison between Real Project and Simulation Data

We collected both simulated and real-world data for various variables, and Figure 6 illustrates a daily profile of chiller plant cooling power, for instance. Overall, the simulated data closely aligns with the real-world data, demonstrating a strong consistency. Due to the ability to simulate a wider range of operating conditions, the simulated data offers a broader coverage of scenarios. This increased diversity in the simulated conditions allows for a more comprehensive representation of potential system behaviors, enhancing the robustness of the model training and its ability to generalize to different operational contexts.

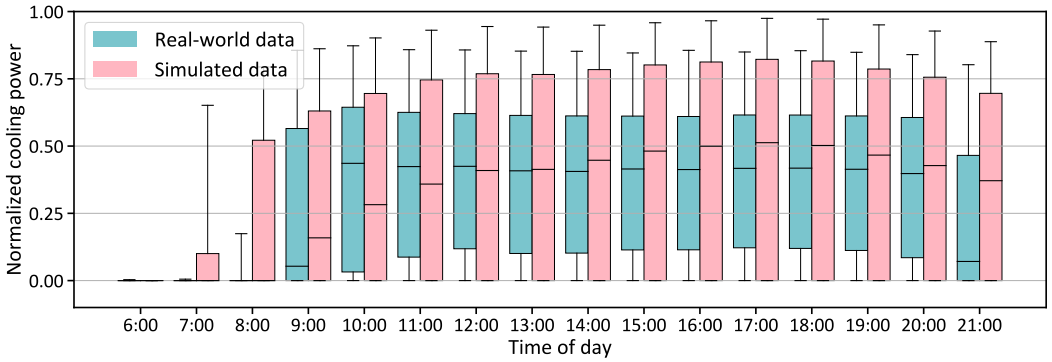

Figure 6: Comparison of a daily profile for chiller plant cooling power between real-world data and simulated data.

C Supplementary Explanation of Network Units and Formulas

C.1 Gated Residual Network (GRN)

The following description of the Gated Residual Network (GRN) is primarily based on the relevant sections from the Temporal Fusion Transformer (TFT) paper [14]. The GRN introduces a gating

mechanism via the Gated Linear Unit (GLU) to regulate the flow of information and selectively pass only the most relevant inputs. This design is critical for handling diverse data inputs effectively. The GRN structure is described by Equations 27-29. The primary input a and the context input c are processed through the Exponential Linear Unit (ELU) activation function, linear transformation, GLU, and layer normalization. Weight matrices W_1, W_2, W_3, and biases b_1, b_2 govern the transformation, providing flexibility through selective non-linear processing.

$$\mathrm{GRN}(a, c) = \mathrm{LayerNorm}(a + \mathrm{GLU}(\eta_1)), \tag{27}$$

$$\eta_1 = W_1\eta_2 + b_1, \tag{28}$$

$$\eta_2 = \mathrm{ELU}(W_2a + W_3c + b_2). \tag{29}$$

The GLU is defined in Equation 30, where X is the input, W_4 and W_5 are learnable weights, b_3 and b_4 are biases, and σ is the sigmoid function. The Hadamard product $\odot$ modulates the GRN's influence on the input a, allowing it to potentially skip processing when the GLU output approaches zero. If no context vector is provided, c is set to zero.

$$\mathrm{GLU}(X) = \sigma(W_4X + b_3) \odot (W_5X + b_4). \tag{30}$$

This modular structure enables the GRN to adapt flexibly to different input types and feature combinations, enhancing the Variable Selection Networks' (VSNs) ability to identify and prioritize key variables efficiently.

C.2 Variable Selection Network (VSN)

The variable selection weights α are computed to determine the contribution of each time-variant feature x_i to the aggregated embedding e^{agg}. This is achieved through a Gated Residual Network (GRN) and a softmax function as shown below:

$$\alpha = [\alpha_1, \ldots, \alpha_i, \ldots, \alpha_m] = \mathrm{Softmax}(\mathrm{GRN}([e_1, \ldots, e_i, \ldots, e_m], c_s)), \tag{31}$$

where c_s is the static covariate encoder and e_i is the embedding vector of feature x_i. The aggregated entity embedding vector e^{agg} is a weighted sum of all the m time-variant variable embeddings:

$$e^{\mathrm{agg}} = \sum_{i=1}^{m} \alpha_i \, \mathrm{GRN}(e_i). \tag{32}$$

VSN can be also used for static feature selection, and Figure 7 presents the VSN architecture, with using GRN.

C.3 Transformer

The self-attention mechanism in Transformer layers enhances the embeddings by considering the relationships between all elements in the input sequence, allowing the model to capture global context and complex dependencies. The mechanism works by calculating a similarity score between each query (Q) and key (K) pair, producing attention weights that reflect the importance of each element in relation to others. These attention weights enable each element to be influenced by other relevant elements in the sequence, leading to a dynamic and context-aware representation.

The self-attention mechanism computes the attention weights for a given set of query, key, and value matrices Q, K, and V as follows:

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{33}$$

where Q, K, and V are the query, key, and value matrices, respectively. d_k is the dimensionality of the key vectors. The term $\frac{QK^T}{\sqrt{d_k}}$ ensures that the dot-product similarity is normalized by the square root of the dimensionality, preventing large values that could make the softmax function too sharp. The softmax function is applied to the similarity scores to generate a probability distribution, which is then used to weight the values in V.

The multi-head attention mechanism allows the model to capture information from multiple representation subspaces. Instead of computing a single attention output, multiple attention heads are

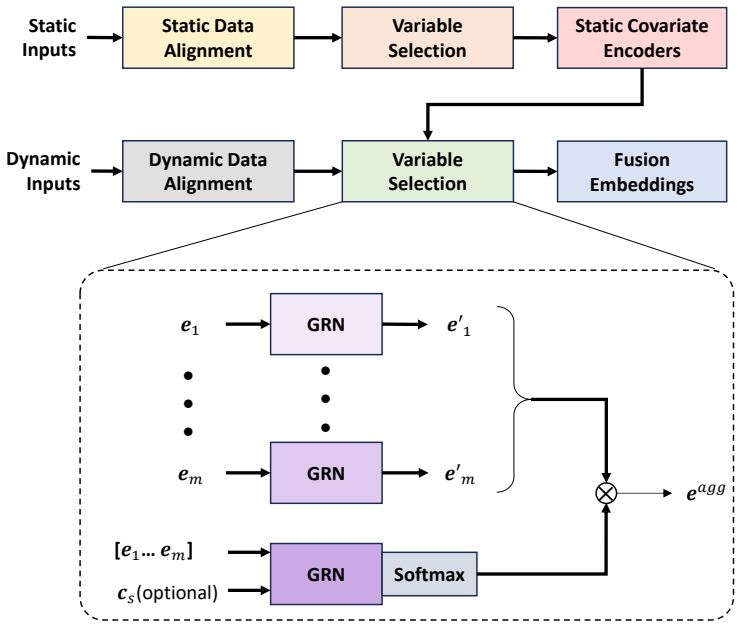

Figure 7: Overview of the entire workflow of the Fusion Layer, where both static and dynamic data pass through two Variable Selection Networks (VSN) with distinct parameters. The static features are selected by themselves, while dynamic data is filtered based on selected static covariates. The calculation mechanism of the VSN is also depicted in the diagram.

computed in parallel, and their results are concatenated and projected back to the original space. The multi-head attention mechanism is defined as:

$$\text{MultiHead}(Q, K, V) = [H_1 \oplus \cdots \oplus H_h \oplus \cdots \oplus H_H]W_H, \tag{34}$$

where H_h represents the output of the h-th attention head, computed as:

$$H_h = \text{Attention}(QW_h^Q, KW_h^K, VW_h^V), \tag{35}$$

and W_h^Q, W_h^K, and W_h^V are learned weight matrices for the query, key, and value matrices, respectively, for the h-th head. The symbol $\oplus$ denotes concatenation, meaning the outputs from all attention heads are concatenated into a single vector. W_H is a learned weight matrix that projects the concatenated output back into the model's desired output dimension.

After the multi-head attention step, a feed-forward network (FFN) is applied to introduce non-linearity. The FFN consists of two fully connected layers with a ReLU activation function applied between them. This non-linearity enables the model to capture more complex relationships and dependencies within the data.

Thus, the combination of self-attention and multi-head attention allows the Transformer model to focus on different parts of the input sequence simultaneously, creating a more dynamic and contextually aware representation, especially useful for tasks involving long-range dependencies and complex sequence data.

C.4 Intra-relation Aggregation

To preserve graph heterogeneity and enable fine-grained relation modeling, we perform relation-specific neighborhood aggregation using distinct BiLSTM encoders for each relation type.

At time step t, the system is represented as a heterogeneous graph $\mathcal{G}_t = (V, E, R)$, where R denotes the set of edge relation types. For each node $v_i \in V$ and relation $r_\ell \in R$, we aggregate temporal embeddings $h_{j,t}^{\text{temp}}$ from neighbors $v_j \in N_\ell(v_i)$ using:

$$h_{i,\ell}^{\text{agg}}(t) = \frac{1}{|N_\ell(v_i)|} \sum_{v_j \in N_\ell(v_i)} \text{BiLSTM}_\ell(h_{j,t}^{\text{temp}}), \tag{36}$$

Unlike HetGNN, which shares encoders across neighbor types, we assign a distinct BiLSTM per relation type r_ℓ, allowing the model to disentangle heterogeneous physical or logical interactions. For example, a room might be connected to others via either airflow or control signals—relations that are semantically different and thus require different encoding strategies.

C.5 Inter-relation Aggregation

To integrate information from multiple relation types, we adopt a multi-head attention mechanism over the aggregated embeddings $h_{i,\ell}^{\text{agg}}(t)$. For each attention head $k = 1, \ldots, K$, attention coefficients α_ℓ^k are computed as:

$$\alpha_\ell^k = \text{softmax}\Big(\text{LeakyReLU}\left(a^{k\top}[W^k h_{i,t}^{\text{temp}} \| W^k h_{i,\ell}^{\text{agg}}(t)]\right)\Big), \tag{37}$$

where $W^k \in \mathbb{R}^{d' \times d}$ is a learnable projection matrix and $a^k \in \mathbb{R}^{2d'}$ is a shared attention vector for the k-th head. The final graph-based embedding for node v_i is:

$$h_{i,t}^{\text{graph}} = \frac{1}{K} \sum_{k=1}^{K} \sum_{\ell=1}^{L} \alpha_\ell^k W^k h_{i,\ell}^{\text{agg}}(t), \tag{38}$$

This fusion mechanism allows the model to assign adaptive weights to different relation types per attention head, enabling robust modeling of heterogeneous dependencies. Compared to early fusion approaches, this method provides enhanced flexibility and improved representation quality for nodes participating in multi-relational contexts.

D Model Version Comparison

This section presents a systematic comparison of different model variants for time-series forecasting in complex building operation systems. All models are trained on the MBS dataset, using the proposed HGTFT architecture. The resulting pretrained model, specialized for the building domain, is termed BOSG (Building Operation System Generator). We explore a range of model configurations by varying embedding dimensions, network depth, and overall parameter count to analyze trade-offs between predictive performance, model size, and training efficiency.

D.1 Embedding Dimension Adjustment

We tested four embedding dimensions (64, 128, 256, and 512), while keeping the architecture constant: one temporal layer, one graph layer, and two additional temporal layers. Results in Table 5 show that 256-d offers a strong trade-off between accuracy and efficiency. Although 512-d provides marginal MSE improvements, the parameter increase is substantial, with limited performance benefit.

D.2 Model Layer Adjustment

We compared multiple network layer configurations, modifying the order and count of temporal and graph layers (see Table 6). Results indicate that placing a temporal layer before the graph layer is essential for capturing temporal context prior to modeling inter-object relations. Additional temporal layers after the graph layer further improve performance, but benefits plateau beyond two layers.

Table 5: Performance comparison of various model configurations with different embedding dimensions.

Embedding dimension	Model size	MSE	RCS	CRS	FDS
64-d	22,241,773	0.0098	0.0168	0.434	0.491
128-d	81,437,154	0.0059	0.0045	0.396	0.448
256-d	310,800,689	0.0027	**0.0012**	**0.312**	**0.405**
512-d	1,173,418,849	**0.0026**	**0.0012**	0.320	0.413

Table 6: Performance comparison of various model configurations with different network layer architectures.

Layer configuration	Model size	MSE	RCS	CRS	FDS
Graph+Temporal	197,075,249	0.0068	0.0025	0.487	0.536
Temporal+Graph	197,075,249	0.0056	0.0021	0.469	0.480
Temporal+Graph$\times$2	213,697,841	0.0053	0.0020	0.454	0.477
(Temporal+Graph)$\times$2	270,560,561	0.0039	0.0018	0.413	0.439
(Temporal+Graph)$\times$3	344,045,873	0.0033	0.0015	0.375	0.414
Temporal+Graph+Temporal	253,937,969	0.0037	0.0016	0.395	0.468
Temporal$\times$2+Graph+Temporal	310,800,689	0.0036	0.0016	0.386	0.442
Temporal$\times$2+Graph+Temporal$\times$2	367,663,409	0.0028	0.0013	0.306	0.399
Temporal+Graph+Temporal$\times$2	310,800,689	0.0027	**0.0012**	0.312	0.405
Temporal+Graph+Temporal$\times$3	367,663,409	**0.0026**	**0.0012**	**0.304**	**0.397**

D.3 Scaling Study and Model Variants

We conducted a scaling study on the BOSG model to investigate the relationship between model size, computation, and forecasting performance. Four BOSG configurations were trained with parameter sizes of 20M, 80M, 310M, and 1.26B, each using 30K iterations and a fixed global batch size of 64. All model variants adopted 8 attention heads and incorporated up/down projection layers to enhance feature representation. Their architectural details and evaluation results are summarized in Table 7. As model size increased, the primary forecasting metric (MSE) consistently decreased from 0.0107 to 0.0025, with notable gains up to 310M parameters. However, performance improvement between the 310M and 1.26B models was marginal, indicating diminishing returns at larger scales.

To better understand compute-performance efficiency, we saved model checkpoints at specific FLOPS intervals during training and plotted the resulting MSE values on a log scale. As shown in Figure 8, training performance improved with increasing computational budget, although the rate of improvement flattened beyond the 310M model. All experiments were conducted on a high-performance system consisting of eight NVIDIA A800 GPUs (80GB memory each), providing 3456 tensor cores in total. This setup enabled efficient parallel training, with the largest model (1.26B) completing 30K iterations in approximately three days. These findings provide practical guidance for compute-optimal scaling in time-series modeling.

Table 7: Performance comparison of BOSG model variants with varying parameter sizes and configurations.

Version	Params	Embedding	Layer configuration	MSE	RCS	CRS	FDS
20M	19,164,316	64-d	Graph+Temporal	0.0107	0.0184	0.496	0.519
80M	77,202,922	128-d	Temporal+Graph+Temporal	0.0073	0.0055	0.435	0.481
310M	310,800,689	256-d	Temporal+Graph+Temporal$\times$2	0.0027	**0.0012**	0.312	**0.405**
1.26B	1,258,271,153	512-d	Temporal+Graph+Temporal$\times$3	**0.0025**	**0.0012**	**0.307**	0.416

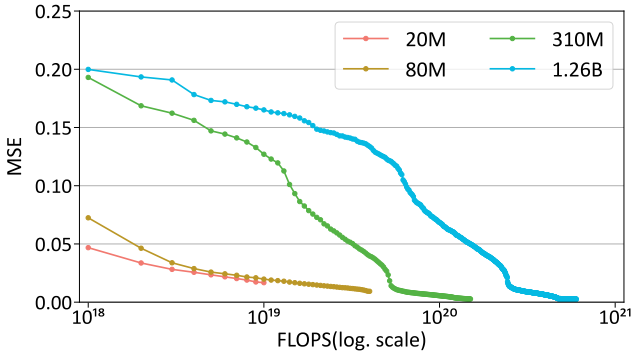

Figure 8: Training MSE vs. FLOPS (log scale) for different BOSG model sizes (20M, 80M, 310M, 1.26B) on the SSL masked time-series modeling task.

E Self-supervised Learning

E.1 Loss Function Details for Self-supervised Learning

The following describes the loss functions employed in our self-supervised learning (SSL) tasks, aimed at ensuring clarity and reproducibility. For masked time-series modeling, the reconstruction error is quantified using the Mean Squared Error (MSE) as follows:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{F_i}\sum_{f=1}^{F_i}\left(\frac{1}{M_{i,f}}\sum_{t=1}^{M_{i,f}}\left(y_{i,f}(t)-\hat{y}_{i,f}(t)\right)^2\right)\right), \tag{39}$$

where N represents the number of nodes, F_i denotes the number of features for the i-th node, and $M_{i,f}$ is the number of masked time points for each feature.

For the graph-based task, the Binary Cross-Entropy (BCE) loss function is utilized to evaluate the classification accuracy of edge predictions, defined as:

$$\text{BCE} = -\frac{1}{N_r}\sum_{i=1}^{N_r}\left(r_i\log(\hat{r}_i)+(1-r_i)\log(1-\hat{r}_i)\right), \tag{40}$$

where N_r represents the number of samples, $\hat{r}_i$ is the predicted relation, and r_i denotes the true relation value.

E.2 Model Training Experiments for Self-supervised Relationship Learning Task

We conducted experiments for self-supervised relationship learning task with the HGTFT model to identify which network layers are essential to update and which can remain fixed. Additionally, we evaluated the prediction results when the parameters of the task-specific linear transformation layer were either initialized randomly without updates or jointly updated alongside HGTFT. Further, we examined the effect of initializing HGTFT parameters either randomly or using pre-trained weights from a masked time-series modeling task. The results of these validation experiments are summarized in Table 8.

The experimental results revealed that updating the network layers responsible for the temporal and graph embeddings is crucial for task performance. Additionally, reusing the pre-trained parameters from the masked time-series modeling task provided a significant improvement over random initialization. Interestingly, the task-specific linear output layer primarily acted as a dimensionality transformation and had minimal impact on the prediction results. Based on these observations, we determined that the optimal approach involves initializing the base HGTFT model parameters from the trained masked time-series modeling task, updating only the temporal and graph embeddings, and leaving the task-specific linear output layer randomly initialized and fixed during training.

Table 8: Experimental results of masked edge modeling for various model update approaches.

Case No.	HGTFT update layer	Task NW	Initialization	loss (BCE)
Case 1	node, temporal, graph	Update	Random	0.34
Case 2	temporal, graph	Update	Random	0.35
Case 3	graph	Update	Random	0.42
Case 4	temporal, graph	w/o update	Random	0.35
Case 5	temporal, graph	w/o update	Masked time-series modeling	**0.28**

E.3 Training Pipeline for Self-Supervised Learning

In our self-supervised learning approach, we prioritized the masked time-series modeling task as the primary objective, with the masked edge modeling task as a secondary target. The goal was to minimize the loss of the masked edge modeling task while ensuring that the loss of the masked time-series modeling task increased by no more than 10% from its optimal value. A series of sequential training experiments were conducted to achieve this balance, and the results are summarized in Table 9.

Table 9: Experiment results for the self-supervised learning pipeline.

Step No.	Masked time-series modeling			Masked edge modeling		
	Task on/off	Starting loss	Ending loss	Task on/off	Starting loss	Ending loss
Step 1	On	1.8421	0.0027	Off	N/A	0.6942
Step 2	Off	0.0027	0.6439	On	0.6942	0.2885
Step 3	On	0.6439	0.0028	Off	0.2885	0.4526
Step 4	Off	0.0028	0.2781	On	0.4526	0.2640
Step 5	On	0.2781	0.0026	Off	0.2640	0.3304
Step 6	Off	0.0026	0.2673	On	0.3304	0.2595
Step 7	On	0.2673	0.0026	Off	0.2595	0.3184

Through a total of seven rounds of alternating training between the two tasks, we observe a consistent decrease in the loss for the masked time-series modeling task before each training session, with little change in the loss after training. In contrast, for the masked edge modeling task, the loss values showed noticeable reductions both before and after training in each round. Notably, the final round of training for the masked time-series modeling task had minimal impact on the graph relationship prediction, suggesting that the model had converged and further training on this task no longer significantly affected the performance of the masked edge modeling task.

F Supervised Learning

F.1 Supervised Learning subtask model

Each subtask shares a unified decoder structure (see Figure 9), where masked attention connects historical embeddings to future targets. GRN blocks and lightweight dense projections are included for stable adaptation. Fine-tuning is performed in two stages: task-level tuning updates only task-specific parameters, while project-level tuning adjusts the dense head to align with limited real-world data, preserving general representations learned during pretraining.

F.2 Supervised Learning Training Task

Forecasting tasks in multiphysics systems exhibit substantial diversity due to the heterogeneity of entities, variable types, and interaction structures. To capture this complexity, we construct a suite of supervised learning tasks based on scenario-specific interaction topologies. Each scenario is represented as a heterogeneous graph comprising distinct physical entities (e.g., thermal zones, fluid circulation units, environmental sensors) and their relationships, as illustrated in Figure 10 for scenario 3.3.

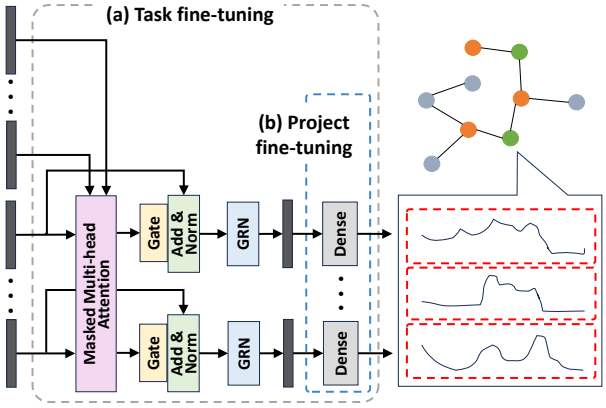

Figure 9: Model structure for a typical prediction subtask with two fine-tuning phases.

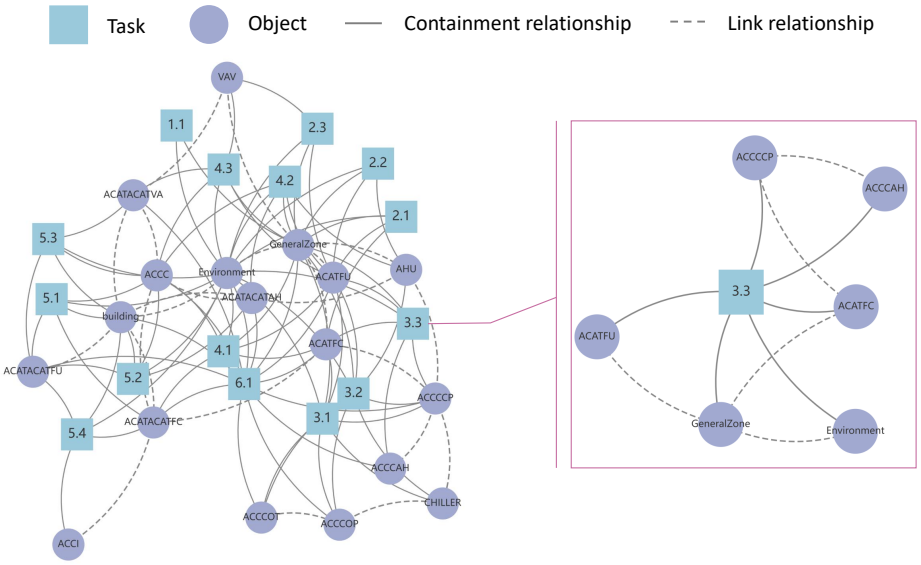

Figure 10: Topology of tasks and entity types in a building multiphysics system. Each scenario defines a specific combination of interconnected entities, identified by a unique scenario ID. Highlighted example 3.3 includes Environment, General Zone, Chiller, Chilled/ Cooling Water Pumps (ACCCCP/ACCCOP), Cooling Tower (ACCCOT), Fan Coil Unit (ACATFC), and Supply Air Fan (ACATFU).

Beyond structural diversity, variations in variable availability across entities further contribute to task differentiation. We first define original tasks by selecting strongly correlated entities and predicting all of their dynamic variables for future time points. Derived tasks are then generated by selectively masking or revealing subsets of variables in the future, simulating diverse observability conditions. An example of such task construction is provided in Table 4.

F.3 Mean Square Error for Supervised Learning

The accuracy loss, denoted as L_{MSE}, is quantified using the Mean Squared Error (MSE) across all entities for each task, as formally defined below:

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \left(y_i(t, T_{\text{future}}) - \hat{y}_i(t, T_{\text{future}}) \right)^2 , \tag{41}$$

where N represents the number of entities, which may vary across different tasks. The terms $y_i(t, T_{\text{future}})$ and $\hat{y}_i(t, T_{\text{future}})$ refer to the true and predicted values, respectively, for the time period from $t+1$ to $t+T_{\text{future}}$, corresponding to all dynamic prediction features of the i-th entity. For the sake of brevity and clarity, the feature dimension is omitted from the formula.

F.4 Reasonableness Checks Score

In complex physical systems, time series predictions must not only achieve numerical accuracy but also respect fundamental physical laws and operational constraints. We propose the *Reasonableness Checks Score* (RCS) as an auxiliary evaluation metric to quantify the degree to which predicted values conform to domain-specific physical expectations. Rather than being limited to any particular field, the RCS framework is designed to be modular and extensible, supporting multi-domain constraints across various physical and engineered systems.

To structure this assessment, we categorize reasonableness checks into four generalized dimensions:

1. **Physical State Bounds:** Core physical quantities (e.g., temperature, pressure, flow rate, power) should remain within known feasible or safe ranges, derived from empirical knowledge or physical laws.
2. **Energy and Resource Balance:** Energy usage, mass flow, or material consumption should be consistent with input-output relationships and operational schedules. Sudden discontinuities or unrealistic surges may indicate violations of conservation principles or faulty control.
3. **System Operating Constraints:** Devices or subsystems should operate in valid configurations, respecting timing constraints, activation conditions, and logical dependencies (e.g., cooling should not activate when the system is already below the lower threshold).
4. **Inter-Component Consistency:** Multiple subsystems interacting within the same environment should exhibit consistent behavior. For example, responses to a shared external stimulus should not contradict each other or physical causality.

Each reasonableness check can be modeled as a differentiable function that penalizes violations of soft physical constraints. The total RCS loss is defined as:

$$L_{\text{RCS}} = \sum_{k=1}^{K} g_k\left(\hat{y}_k(t, T_{\text{future}})\right), \tag{42}$$

where $g_k(\cdot)$ denotes the k-th check function applied to the predicted output $\hat{y}_k(t, T_{\text{future}})$, and K is the total number of checks relevant to the task.

Example 1: Bounded Range Check. For a physical variable $\hat{y}(t)$ constrained within a range $[y_{\min}, y_{\max}]$, the penalty term can be formulated as:

$$g_{\text{range}}(\hat{y}(t)) = \lambda \cdot \left[\max\left(0, \hat{y}(t) - y_{\max}\right)^2 + \max\left(0, y_{\min} - \hat{y}(t)\right)^2\right], \tag{43}$$

where λ is a weighting coefficient controlling the penalty strength at each time point t.

Example 2: Energy Conservation Check. In multi-physical systems, the principle of energy conservation often serves as a key constraint. For instance, in a thermal process involving heat exchange, the heat entering a system at time t should approximately equal the sum of the heat leaving the system and the internal losses, i.e., $Q_{\text{in}}(t) \approx Q_{\text{out}}(t) + Q_{\text{loss}}(t)$. To enforce this physical constraint on predicted outputs, we define the energy conservation check function as:

$$g_{\text{energy}}(t) = \gamma \cdot \left(\hat{Q}_{\text{in}}(t) - \hat{Q}_{\text{out}}(t) - Q_{\text{loss}}(t)\right)^2, \tag{44}$$

where $\hat{Q}_{\text{in}}(t)$ and $\hat{Q}_{\text{out}}(t)$ denote the predicted input and output energy at time t, and $Q_{\text{loss}}(t)$ is a predefined (or estimated) time-dependent loss term. The scalar γ controls the importance of this check. This function penalizes deviations from the expected energy balance at each timestep, thereby promoting physically consistent predictions.

F.5 Correlation-Based Score

The Correlation-Based Score (CBS) evaluates the statistical correlation between predicted and true values in time-series forecasting by computing the Pearson correlation coefficients for both predicted and true values, determining the deviation between these correlations for each variable pair, and then calculating the loss as the Mean Squared Error (MSE) of these deviations.

The formula for the CBS loss L_{corr} is given by:

$$L_{\text{CBS}} = \frac{1}{L} \sum_{l=1}^{L} \left(|\rho(\hat{y}_i, \hat{y}_j) - \rho(y_i, y_j)|^2 \right), \tag{45}$$

Where L is the number of variable pairs in the prediction task. $\rho(\hat{y}_i, \hat{y}_j)$ is the Pearson correlation coefficient between the predicted values $\hat{y}_i$ and $\hat{y}_j$, and $\rho(y_i, y_j)$ is the Pearson correlation coefficient between the true values y_i and y_j.

F.6 Frequency Domain Similarity

To calculate the similarity between two time-series datasets in the frequency domain, we can use the Fourier Transform to convert both datasets from the time domain to the frequency domain and then compare their frequency components. The steps are stated as following:

1. **Fourier Transform:** Apply the Fourier Transform to each time series to obtain the amplitude and phase spectra. Let $A_X(f)$ and $\theta_X(f)$ be the amplitude and phase of the first time-series data across frequencies f. Similarly, $A_Y(f)$ and $\theta_Y(f)$ represent the amplitude and phase of the second time series.
2. **Amplitude Cosine Similarity:** Define the cosine similarity for the amplitude spectra of the two time-series datasets as follows:

$$S_{\text{amp}} = \frac{\sum_{f=1}^{N} A_X(f) \cdot A_Y(f)}{\sqrt{\sum_{f=1}^{N} A_X(f)^2} \cdot \sqrt{\sum_{f=1}^{N} A_Y(f)^2}} \tag{46}$$

   where N is the number of frequency components. This metric evaluates the similarity in amplitude between the two datasets.

3. **Phase Cosine Similarity:** Define the cosine similarity for the phase spectra by converting the phase angles into their respective sine and cosine components:

$$S_{\text{phase}} = \frac{\sum_{f=1}^{N} (\cos(\theta_X(f)) \cdot \cos(\theta_Y(f)) + \sin(\theta_X(f)) \cdot \sin(\theta_Y(f)))}{\sqrt{\sum_{f=1}^{N} (\cos(\theta_X(f))^2 + \sin(\theta_X(f))^2)} \cdot \sqrt{\sum_{f=1}^{N} (\cos(\theta_Y(f))^2 + \sin(\theta_Y(f))^2)}} \tag{47}$$

   This metric evaluates the alignment of phase angles between the two time series.

4. **Combined Frequency Domain Similarity:** Finally, define the combined frequency domain similarity S_{freq} using a weighted sum of the amplitude and phase similarities:

$$S_{\text{freq}} = \alpha S_{\text{amp}} + \beta S_{\text{phase}} \tag{48}$$

   where α and β are weights that can be adjusted based on the relative importance of amplitude and phase similarity. This combined metric S_{freq} captures both amplitude and phase alignment, offering a comprehensive measure of similarity between the two time-series datasets in the frequency domain. The loss for Frequency Domain Similarity (FDS), L_{FDS}, is $1 - S_{\text{freq}}$.

F.7 Supervised Learning Pipeline

The supervised learning pipeline is organized into five stages to enhance the foundation model's ability to aggregate and represent information, building on the self-supervised phase and improving its applicability to generalizable time-series prediction tasks. The primary objective is to enhance the

Table 10: Supervised learning pipeline with loss weights.

Stage No.	Foundation model update	Task model update	Learning rate	a_1	a_2	a_3	a_4
Stage 1	N	Y	0.1	1	0	0	0
Stage 2	Y	Y	0.01	0.5	0.1	0.2	0.2
Stage 3	Y	N	0.005	0.7	0.1	0.1	0.1
Stage 4	Y	N	0.003	0.7	0.3	0	0
Stage 5	Y	N	0.001	0.5	0.5	0	0

adaptability and representational capacity of the foundation model, rather than focusing solely on maximizing accuracy for individual time-series subtasks. Each stage refines a specific aspect of the model, as summarized in Table 10.

In Stage 1, the parameters of the foundation model, initialized from the self-supervised phase, are frozen, with only the task-specific parameters being updated. This allows the model to quickly adapt to a reasonable accuracy range, using a learning rate of 0.1, while prioritizing the MSE loss function.

In Stage 2, both the foundation model and task-specific models are jointly trained, with the learning rate gradually decaying from 0.1 to 0.01. This stage aims to improve prediction accuracy and gradually bring it closer to optimal performance. The loss weights are adjusted to strike a balanced consideration of the different loss functions.

In Stages 3, 4, and 5, the task-specific parameters are frozen, and the foundation model is further refined to enhance generalization capability. The learning rate is progressively reduced to 0.005, 0.003, and 0.001, respectively. During these stages, the loss weights are adjusted to refine model performance. In Stage 3, the focus is on improving accuracy with minimal adjustments to the consistency and rationality losses. In Stages 4 and 5, the loss weights are updated to place greater emphasis on L_{RCS}, promoting improved rationality and domain-specific reasoning.

Data is allocated across the five stages following a 1:3:4:1:1 ratio. The majority of the data is utilized during the second and third stages for joint training and generalization, while the final stages concentrate on fine-tuning the foundation model for improved rationality and consistency.

G Baseline Methods Selection

To evaluate the performance of HGTFT across zero-shot and few-shot forecasting tasks, we compare it against diverse baselines, including classic models (No LMs), time-series large models (Time LMs), and large language model-based methods (LLMs). These methods differ in how they handle input modalities such as time-series (TS), static metadata (Static), and graph structure (Graph), as detailed in Table 11. For each method, we selected the most capable open-source version available to ensure a fair comparison.

Below, we provide an overview of the selected baseline methods and their respective adaptations to our setting:

- **LSTM** and **Autoformer**: Forecast each variable independently, without incorporating static or relational information. Their outputs are aggregated through post-processing to construct full multivariate predictions.
- **TFT**: Combines multivariate time-series data with static features to perform object-level forecasting. It supports variable selection and interpretable attention mechanisms but does not model inter-object dependencies.
- **HTGNN**: Utilizes graph-structured time-series inputs, leveraging the relationships between objects to perform dynamic variable forecasting in a heterogeneous setting.
- **STD-MAE**: Utilizes graph-structured time-series inputs in a homogeneous setting, where the system is decomposed into multiple homogeneous subgraphs. Each subgraph is modeled independently to capture localized spatial-temporal patterns, and the predictions are subsequently aggregated to form the overall system-level forecast.

- **TimesFM** and **MOIRAI**: Encode each object type's time-series data sequentially, forecasting each variable independently. These models do not utilize static or graph information; instead, multivariate predictions are obtained by batching univariate forecasts.
- **LLMTime** and **Time-LLM**: Process multiple object instances simultaneously using only time-series data. These LLM-based models do not account for static metadata or inter-instance relationships, but benefit from large-scale pretraining and context-aware generation.

Table 11: Baseline methods summary.

Method	Input Type	Category	Model Version
LSTM	TS	No LM	—
Autoformer	TS	No LM	—
TFT	TS, Static	No LM	—
HTGNN	TS, Graph	No LM	—
STD-MAE	TS, Graph	No LM	—
TimesFM	TS	Time LM	200M
MOIRAI	TS	Time LM	1.1-R-large
LLMTime	TS	LLM	LLaMA-2 70B
Time-LLM	TS	LLM	LLaMA 7B

In recent years, there has been a large number of work focusing on spatial-temporal forecasting in relatively simple settings involving homogeneous object types and graph structures. Although these works differ from the problem definition and setting in our study, we include several widely recognized spatial-temporal forecasting algorithms from the past 4 years for a more comprehensive comparison. We evaluate their performance on four standard datasets: PEMS04, PEMS08, COVID-19 (JHU), and COVID-19 (NYT). As shown in Table 12, while recent methods continue to make marginal improvements in these benchmarks, the performance gap is narrowing. This highlights a critical limitation: the lack of methods and datasets capable of handling more complex scenarios. Addressing this gap is the primary motivation of our work.

Table 12: Performance comparison on PEMS04, PEMS08, COVID-19 (JHU), COVID-19 (NYT) datasets. Best results are in **bold**, second best are underlined.

Model	PEMS04		PEMS08		COVID-19 (JHU)		COVID-19 (NYT)	
	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE
LSTM [1]	32.48 ± 0.38	55.51 ± 0.74	24.98 ± 0.38	41.71 ± 0.43	122.42 ± 1.41	232.11 ± 3.51	70.59 ± 0.85	139.18 ± 1.71
Autoformer [49]	32.39 ± 0.43	53.19 ± 0.75	25.56 ± 0.34	41.65 ± 0.44	115.77 ± 1.08	198.67 ± 2.48	62.37 ± 0.76	133.35 ± 1.68
TFT [14]	31.32 ± 0.35	48.37 ± 0.67	24.63 ± 0.36	39.74 ± 0.42	121.81 ± 1.43	261.77 ± 3.20	71.36 ± 0.84	158.93 ± 2.30
STformer [21]	31.69 ± 0.48	55.70 ± 0.71	24.91 ± 0.44	43.23 ± 0.59	72.42 ± 0.90	166.86 ± 2.13	49.63 ± 0.64	123.01 ± 1.51
TimesFM [10]	32.57 ± 0.43	55.94 ± 0.68	23.93 ± 0.38	42.41 ± 0.56	99.75 ± 1.10	216.63 ± 2.99	57.07 ± 0.72	113.45 ± 1.62
MOIRAI [9]	33.31 ± 0.45	55.51 ± 0.72	24.03 ± 0.32	42.49 ± 0.56	105.74 ± 1.26	234.24 ± 2.70	81.05 ± 0.96	134.57 ± 1.94
LLMTime [30]	33.69 ± 0.52	52.49 ± 0.76	26.68 ± 0.40	43.94 ± 0.44	115.37 ± 1.22	216.74 ± 2.71	72.24 ± 0.90	157.31 ± 1.87
Time-LLM [32]	32.23 ± 0.40	52.18 ± 0.67	27.74 ± 0.48	40.01 ± 0.44	95.01 ± 1.13	201.14 ± 2.68	83.17 ± 1.01	146.45 ± 2.02
ASTGCN [35]	23.46 ± 0.27	34.88 ± 0.59	17.91 ± 0.31	28.80 ± 0.47	58.10 ± 0.61	109.14 ± 1.91	33.71 ± 0.48	93.55 ± 1.37
STGCN [52]	21.72 ± 0.34	34.61 ± 0.57	18.73 ± 0.35	28.05 ± 0.50	52.94 ± 0.63	110.63 ± 1.68	37.25 ± 0.44	88.62 ± 1.27
STSGCN [24]	21.26 ± 0.28	34.42 ± 0.50	17.86 ± 0.32	27.45 ± 0.45	53.19 ± 0.55	111.51 ± 1.65	34.58 ± 0.46	89.42 ± 1.31
STFGNN [53]	19.24 ± 0.25	31.18 ± 0.55	16.76 ± 0.29	25.74 ± 0.44	51.45 ± 0.56	101.48 ± 1.72	33.00 ± 0.48	82.17 ± 1.38
STGODE [54]	21.63 ± 0.31	33.30 ± 0.49	16.14 ± 0.31	25.46 ± 0.45	56.84 ± 0.66	106.01 ± 1.66	32.09 ± 0.44	81.72 ± 1.33
STNorm [55]	19.07 ± 0.28	31.91 ± 0.52	15.05 ± 0.27	25.64 ± 0.41	46.68 ± 0.53	99.34 ± 1.58	30.59 ± 0.40	80.32 ± 1.21
DSTAGNN [56]	19.87 ± 0.33	30.80 ± 0.53	15.95 ± 0.32	24.53 ± 0.38	50.45 ± 0.57	100.51 ± 1.57	31.49 ± 0.46	76.50 ± 1.31
HTGNN [50]	21.01 ± 0.37	36.44 ± 0.56	18.22 ± 0.38	27.04 ± 0.48	46.24 ± 0.51	102.73 ± 1.56	31.16 ± 0.49	75.98 ± 1.29
PDFormer [57]	18.60 ± 0.29	29.94 ± 0.51	**12.82 ± 0.26**	22.62 ± 0.35	46.57 ± 0.58	93.48 ± 1.21	28.69 ± 0.44	71.70 ± 1.12
STAEformer [58]	18.62 ± 0.30	**29.65 ± 0.44**	12.97 ± 0.26	24.21 ± 0.35	47.58 ± 0.59	96.87 ± 1.34	**24.62 ± 0.41**	77.43 ± 1.30
STD-MAE [51]	**17.85 ± 0.27**	29.72 ± 0.44	13.67 ± 0.28	**22.62 ± 0.36**	47.75 ± 0.60	**92.62 ± 1.27**	26.69 ± 0.45	72.98 ± 1.21
HGTFT (Ours)	19.94 ± 0.34	32.16 ± 0.54	16.43 ± 0.34	25.08 ± 0.41	**41.54 ± 0.44**	94.38 ± 1.20	25.69 ± 0.42	**65.64 ± 1.04**

H Normalization Methods

To evaluate the effectiveness of different normalization methods, using MSE directly on normalized data is not appropriate, as each method applies a unique scaling to the variables, which would make MSE comparisons unfair. Instead, we reverse-normalize the variables before calculating the evaluation metrics to ensure a fair comparison of methods. However, the diverse ranges of the original variables after reverse normalization pose challenges in balancing weights across variables. To address this, we focus on key variables from the training tasks and compute statistical metrics

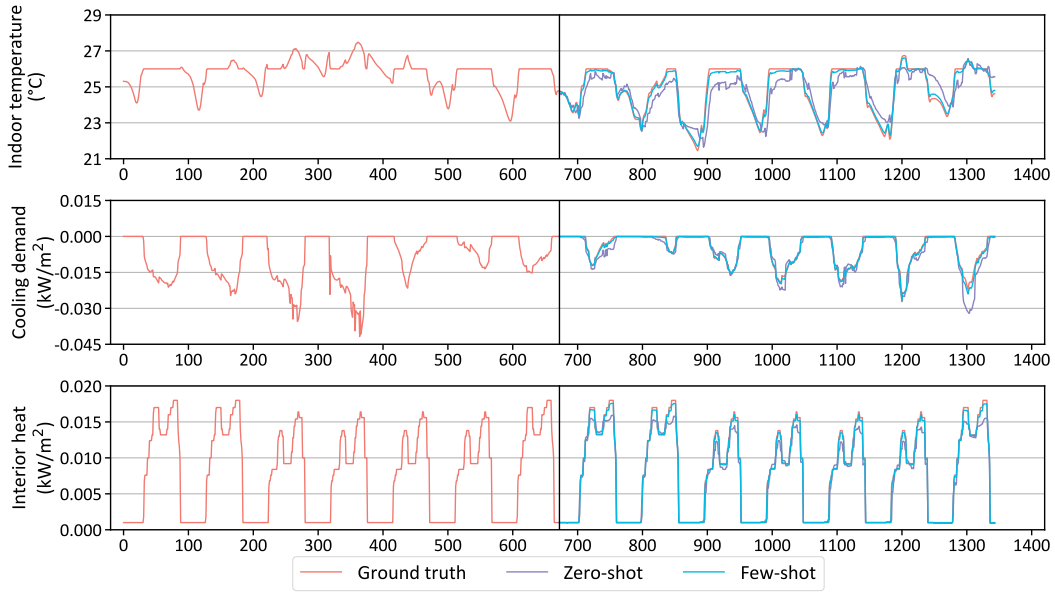

Figure 11: Visualization of time-series forecasting for key variables of a room object, predicting the next 7 days based on the past 7 days. Predictions include zero-shot and few-shot (with one month of fine-tuning data).

for each individually. Their CV-RMSE values are listed in Table 13. As shown, the "Multi-Instance Normalization" method achieves more balanced prediction performance across various variables compared to the other methods.

Table 13: Comparison of average relative errors among various normalization methods. The average error is calculated using the difference between the maximum and minimum values of the actual data for each instance variable as the base for CV-RMSE computation, excluding outlier instances such as devices that have been continuously inactive. The average CV-RMSE is computed for all instances of the same variable type.

Object Type	Typical Variable	Normalization Method		
		Min-Max	Z-score	Multi-Instance
Room	Indoor Temperature	2.1%	**2.0**%	2.9%
	CO_2	**2.4**%	2.5%	2.6%
Chiller	Chilled Water Supply Temperature	7.9%	8.1%	**6.8**%
	Chilled Water Return Temperature	16.3%	14.6%	**11.2**%
	Chilled Water Flow Rate	45.6%	33.9%	**16.8**%
Chilled Water Pump	Operating Power	34.6%	33.7%	**13.6**%
	Flow Rate	39.9%	38.2%	**12.8**%
Cooling Water Pump	Operating Power	42.5%	42.6%	**15.7**%
	Flow Rate	45.5%	48.4%	**16.1**%
Cooling Tower	Leaving Tower Water Temperature	23.0%	24.3%	**7.6**%
	Water Flow Rate	35.3%	33.8%	**15.5**%
Fan Coil Unit	Supply Air Temperature	9.8%	9.2%	**8.3**%
	Return Air Temperature	5.7%	6.1%	**3.6**%
Supply Air Fan	Fresh Air Flow Rate	41.1%	40.7%	**21.3**%

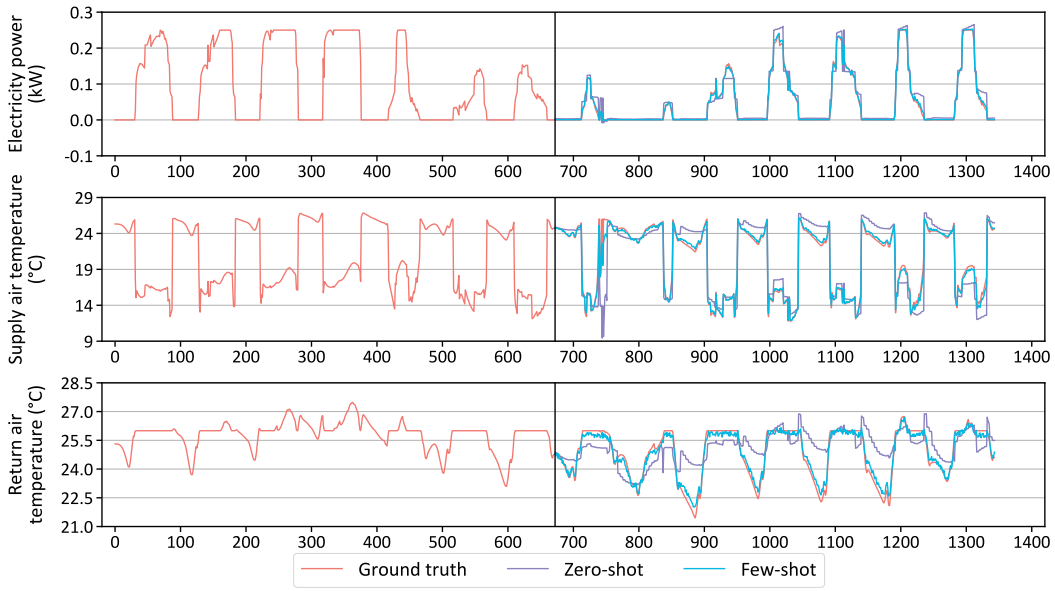

Figure 12: Visualization of time-series forecasting for key variables of a fan coil unit object, predicting the next 7 days based on the past 7 days. Predictions include zero-shot and few-shot (with one month of fine-tuning data).

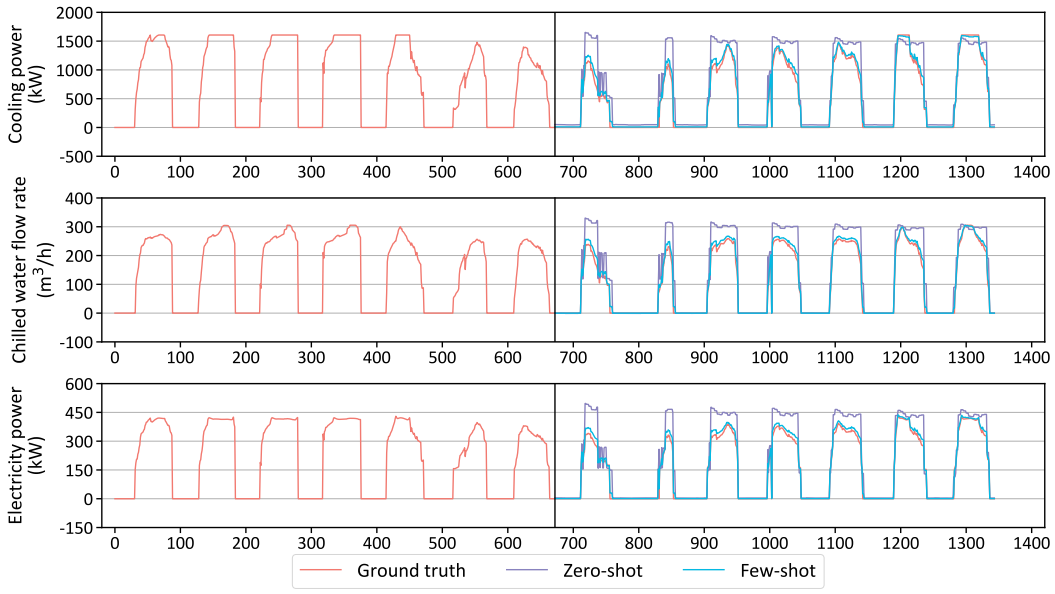

Figure 13: Visualization of time-series forecasting for key variables of a chiller system object, predicting the next 7 days based on the past 7 days. Predictions include zero-shot and few-shot (with one month of fine-tuning data).

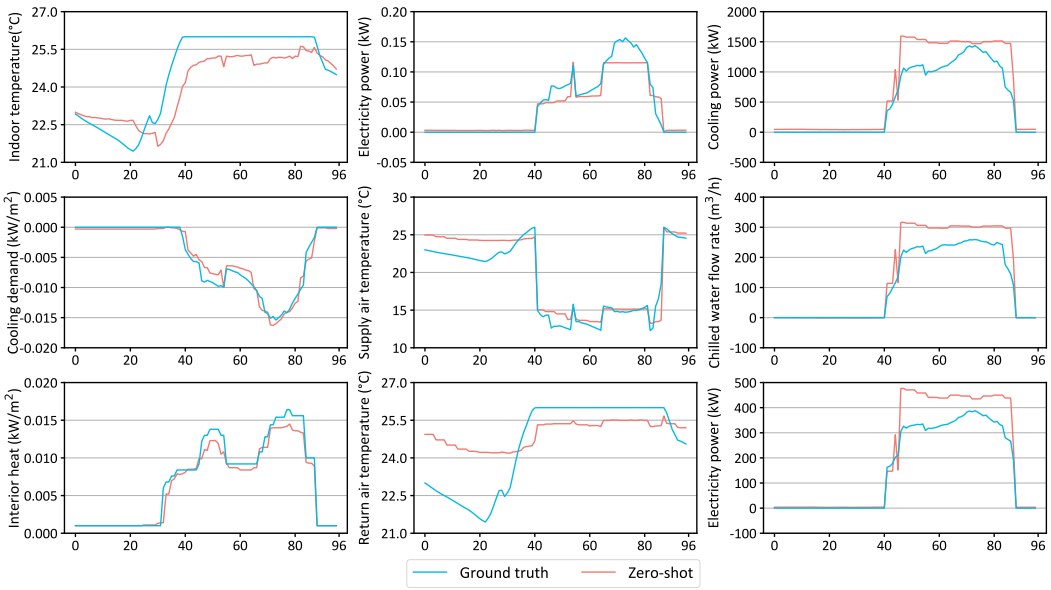

Figure 14: Curves for one day of three related object instances: Room, Fan Coil Unit, and Chiller System, along with their zero-shot prediction results. The left column shows three key variables for the Room object, the middle column for the Fan Coil Unit, and the right column for the Chiller System.

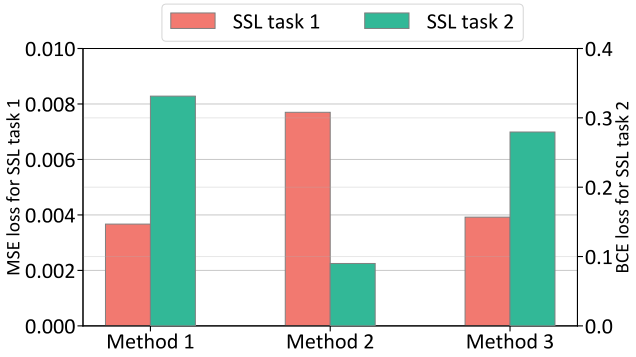

Figure 15: Comparison of three SSL training strategies: separate training, simultaneous training, and alternating training. Task 1 (masked time-series modeling) uses MSE loss, while Task 2 (masked edge modeling) uses BCE loss.

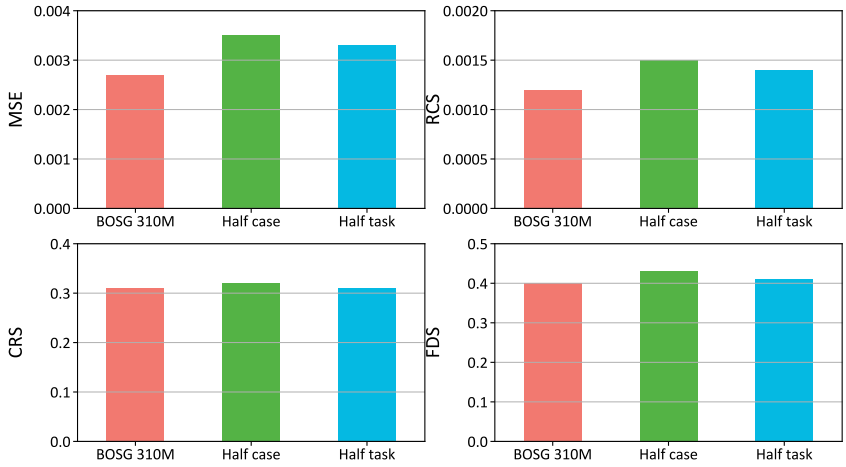

Figure 16: Performance comparison of the BOSG-310M model under three settings: full training, half the number of training cases, and half the number of tasks. Evaluation is based on multiple metrics.

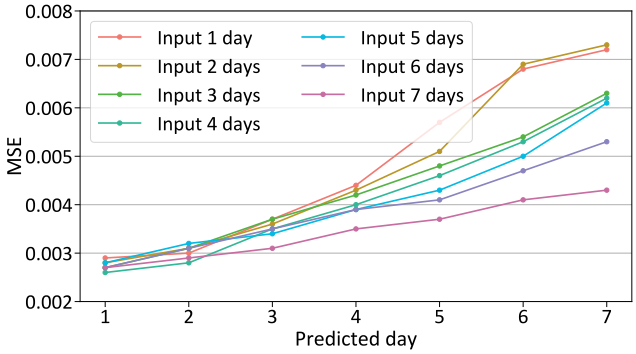

Figure 17: Prediction accuracy for daily rolling forecasts over a 7-day horizon. Each line represents a different input patch length, ranging from 1 to 7 days.

I Time Series Forecasting Visualization

To facilitate a qualitative analysis of the zero-shot and few-shot prediction results based on the BOSG-310M, we present time-series prediction plots for several key variables. The plots illustrate the forecasting performance of the proposed model on three critical objects: room, fan coil unit (FCU), and chiller system, under both zero-shot and fine-tuned conditions.

As shown in Figures 11 to 13, the zero-shot predictions capture the overall trends and patterns for each variable, although the accuracy of the predictions varies across different variables. While the model is able to predict the general shape of the curves, the degree of precision differs, reflecting the inherent challenges of making predictions without prior task-specific fine-tuning. In addition, we present the results of predictions following fine-tuning with one month of data. The improvements are evident, with significantly enhanced accuracy across all variables, particularly in capturing short-term dynamics. However, it is important to note that fine-tuning with a relatively short period of data, although it improves predictions for recent time period and conditions similar to those seen in the fine-tuning phase, may deteriorate predictions for longer time horizons or when faced with highly divergent operational scenarios. To investigate this, we also tested fine-tuning with longer data windows (three months and six months), and found that, overall, predictions for extended timeframes benefited from the use of larger fine-tuning datasets. This suggests that a more extended fine-tuning period helps to mitigate overfitting and ensures better generalization for long-term predictions. However, a key challenge remains: how to fine-tune effectively with limited data while avoiding

overfitting and preserving our foundation model's ability to learn the underlying physical dynamics. This continues to be an area of significant research interest.

We present daily profile curves for multiple dynamic variables of three different types of objects, along with their zero-shot prediction results. In Figure 14, we observe that the temporal relationships between the associated objects are effectively captured. In particular, the predictions for the chiller system and FCU demonstrate that the forecasted surge in cooling power for the Chiller System at time point 42 closely aligns with the predicted supply air temperature of the FCU at the same time. Similarly, the slight decrease in the room's indoor temperature at time point 66 is well-aligned with the small increase in the electricity power consumption of the FCU. In some instances, these temporal relationships are even more pronounced in the predicted data than in the actual observations, highlighting the model's capability to effectively capture interdependencies across various components in the system.

J Additional Results

J.1 Self-supervised Learning Comparison.

We investigate the interaction between two self-supervised learning (SSL) tasks: masked time-series modeling and masked edge prediction. Figure 15 compares three SSL training strategies: (1) training each task independently, (2) simultaneous multi-task training, and (3) our proposed alternating task training. Results show that while simultaneous training impairs the performance of the time-series task, the alternating training method maintains low loss for both tasks, offering a better trade-off between sequence forecasting and structural relation modeling.

J.2 Impact of Task Diversity and Data Quantity.

Leveraging SSL-pretrained weights, we adopt a sequential multi-task learning framework where downstream tasks are optimized one after another. During training, the average task loss consistently decreases across rounds, and the rate of change stabilizes, indicating convergence under the serialized learning schedule. We further conduct ablation studies by halving the number of training tasks and the number of training cases, respectively. As shown in Figure 16, both reductions lead to moderate performance degradation, highlighting the importance of maintaining sufficient task diversity and data coverage for robust generalization.

J.3 Effect of Input Patch Length and Forecasting Horizon.

We evaluate the model's performance across different input and output durations, ranging from 1 to 7 days. Figure 17 presents the results of daily rolling forecasts, where the x-axis denotes the target prediction day and the y-axis represents the corresponding MSE. Each curve corresponds to a different input patch length. The results demonstrate that longer input sequences generally yield improved accuracy, particularly for longer forecasting horizons.

K Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction clearly summarize the paper's contributions, including the HGTFT model, its training paradigm, and validated performance in multiphysics forecasting tasks.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: We have discussed the limitations of our work in Conclusion.

   Guidelines:

   - The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: No theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We provide the model architecture and training methodology in Section 4 and Section 5, with additional implementation and reproducibility details presented in Appendix E and Appendix F.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We have provided anonymized code in the Supplementary Material for review purposes. The code will be publicly released upon acceptance. The dataset used for the main experiments is accessible via an anonymous URL as provided in Section 6.1, and additional public datasets are also described in Section 6.1.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [Yes]

Justification: We have included the training and test details in Appendix E.3, Appendix F.7, and Section 6.3.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We conduct 10 independent runs of each main experiment on the multiphysics system dataset and report the average performance metrics in Table 2. We also show the confidence intervals in Table 12 for the benchmarks.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: We provide computer resources and training time in Appendix D.3.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This research complies with the NeurIPS Code of Ethics in all aspects.

   Guidelines:

   - The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: This work aims to improve time-series forecasting in complex physical systems, such as energy-efficient building management and infrastructure monitoring, potentially contributing to sustainability and operational optimization. While the model may be applied in other domains, care should be taken to ensure responsible deployment, especially regarding data quality and domain-specific safety requirements. We foresee no direct negative societal impacts.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [NA]

Justification: No risks.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All external assets (code, data, models) used in this paper have been properly credited, and their licenses and terms of use are explicitly mentioned and fully respected.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.

- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: New assets, including the datasets and models, are provided via an anonymous link in the paper, while the code can be found in the submitted Supplementary Material. These assets will be made publicly available once the paper is accepted.

    Guidelines:

    - The answer NA means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: This paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: This work doesn't involve research requiring IRB review.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.

- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

    Answer: [NA]

    Justification: This research does not involve the usage of large language models (LLMs) as a core component of the methodology.

    Guidelines:

    - The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.